Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

Language models now write working programs, summarize research papers, and hold conversations that many people find hard to tell apart from human ones. In the last year they have also begun to settle open problems in mathematics, including a conjecture of Paul Erdős that had stood for eighty years (§1).

How these systems work is mostly understood at the level of their parts: a probability distribution over strings, computed by a large network, fitted to text, adjusted with demonstrations and rewards, and sampled at inference time. Why they work, and what they can and cannot do, is much less settled. This course studies the whole stack from first principles and builds a small working language model along the way.

This lecture has four parts:

1. a short history, from the first mathematical models of computation to today's language models, and what the course will build (§§1–2);
2. the vocabulary of artificial intelligence, machine learning, and deep learning (§3);
3. four old questions that these systems make newly concrete: what intelligence is, what it means to learn, what language is, and what meaning is (§§4–7);
4. a hands-on walk through the automatic-differentiation engine that trains every neural model in the course (§8).

1. From computation to language models

Humans stand out among the species around them in one respect above all: they build tools that make their lives better, and they pass those tools on.

For most of history the tools were physical. The control of fire gave warmth, protection, and cooked food. Agriculture fed settled communities. The wheel, the plough, and metal tools multiplied what a person or an animal could move and make. Then the steam engine, electricity, and the internal combustion engine replaced muscle with machine power, and the industrial revolutions that followed reshaped where and how people live (Figure 1, top row).

A second family of tools extends what humans can know. Spoken language lets one person pass knowledge to another. Writing and numerals store it outside any single mind, printing copies it cheaply, and the telegraph, the telephone, and radio carry it across distances almost instantly (Figure 1, bottom row). Knowledge that can be stored and sent can accumulate: each generation starts from what earlier ones wrote down, wherever and whenever they lived.

Electronic computers belong to this second family and add something new. They do not only store and move information; they transform it by following a procedure, at a speed and scale no person can match. Language models are a recent step in this line. They make a wide range of information-processing tasks available through ordinary language.

Landmarks in computation and artificial intelligence

1. **1936: Turing's model of computation.** Alan Turing made "mechanical symbol manipulation" a precise mathematical object: a finite table of rules acting on an unbounded tape. It makes precise what can be computed at all (Figure 2a).
2. **1946 and 1947: ENIAC and the transistor.** The first general-purpose electronic computer was announced in 1946. The transistor, demonstrated at Bell Labs in 1947, later made computers small and cheap (Figure 2b, c).
3. **1950: Turing's imitation game.** Turing replaced the question "Can machines think?" with a test of observable behavior: can an interrogator, conversing by text, tell a machine from a person? (Figure 2d; §4.)
4. **1955 and 1956: the Dartmouth proposal and workshop.** The proposal named the field "artificial intelligence" and conjectured that every aspect of intelligence can be described precisely enough for a machine to simulate it (Figure 2e).
5. **1958: Rosenblatt's perceptron.** A decision rule learned from labeled examples by adjusting numerical weights, and built in hardware as the Mark I Perceptron (Figure 2f; §5).



(a) The control of fire



(b) The wheel



(c) Steam power



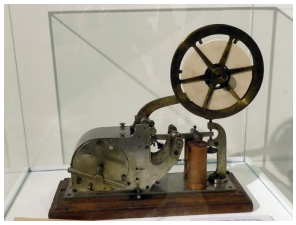
(d) Writing



(e) Numerals, with a zero



(f) Printing



(g) The telegraph

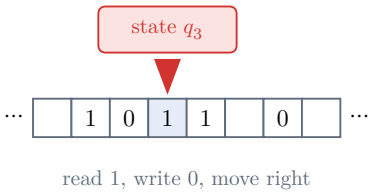
Figure 1. Tools that extend physical ability (top row) and tools that store and move information (bottom row). Photographs from Wikimedia Commons; see the image credits at the end.

6. **1959: Samuel's checkers program.** A program that improved by playing against itself. The title of Arthur Samuel's paper is an early use of the phrase "machine learning" (Figure 2g).
7. **1986: backpropagation.** David Rumelhart, Geoffrey Hinton, and Ronald Williams popularized training networks with hidden layers by following the gradient of a loss, so that a network learns its own intermediate features (Figure 2h; §§5 and 8).
8. **1997: Deep Blue defeats Kasparov.** Deep search, an evaluation function written by human experts, and custom chips beat the world chess champion (Figure 2i).
9. **2012: AlexNet.** A deep network trained on 1.2 million labeled images with graphics processors cut the ImageNet error rate sharply. Learned features replaced features designed by hand (Figure 2j).
10. **2014: generative adversarial networks.** A generator and a discriminator, trained against each other, produce realistic images: an early landmark of neural generative modeling (Figure 2k).
11. **2016: AlphaGo defeats Lee Sedol, 4–1.** Networks learned, from expert games and from self-play, which moves are worth exploring and who is winning; a tree search combined the two (Figure 2l).

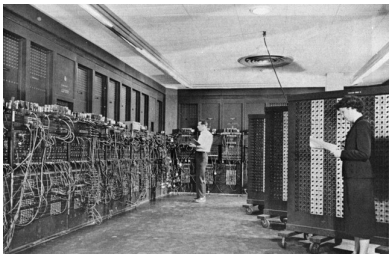
Artificial intelligence, machine learning, deep learning, and generative modeling appear in this list as overlapping fields that developed side by side; §3 defines them. Items 8 and 11 set up a contrast that §4 returns to. Both Deep Blue and AlphaGo search, but Deep Blue's knowledge of chess was written by human experts, while AlphaGo's knowledge of Go was learned. Between items 8 and 9 the list skips fifteen years in which neural networks were out of favor and most of the field turned to statistical methods; §B describes them.

Language models

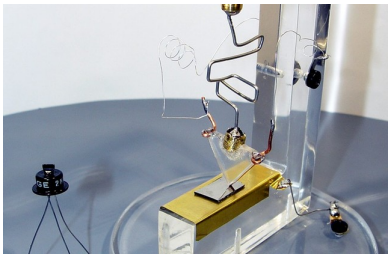
1. **2017: the Transformer.** An architecture built on attention, in which each position of a sequence reads directly from other positions. It was designed for machine translation, as an encoder that reads a sentence in one language and a decoder that writes its translation; the decoder alone, trained to predict the next token, became the basis of language models, and it is the architecture this course implements (Figure 3a).
2. **2018: GPT and BERT.** A Transformer pretrained on unlabeled text, then fine-tuned for each task, beat models built for individual tasks (Figure 3b).
3. **2020: GPT-3.** A model with 175 billion parameters performs new tasks from a prompt containing a few examples, with its weights held fixed (Figure 3c).
4. **2022: ChatGPT.** Post-training with demonstrations and human preferences, together with a conversational interface, made a language model usable as an assistant (Figure 3d).



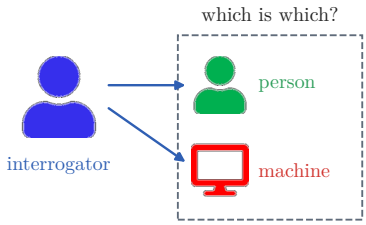
(a) 1936: a Turing machine; a head reads and writes one cell of a tape



(b) 1946: ENIAC



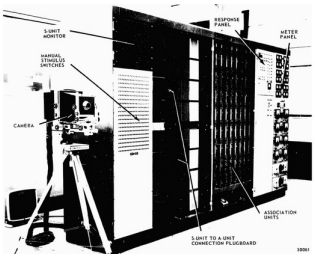
(c) 1947: a replica of the first transistor



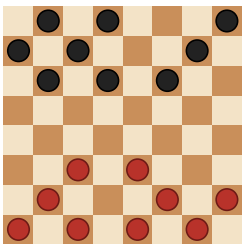
(d) 1950: the imitation game

"Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it."

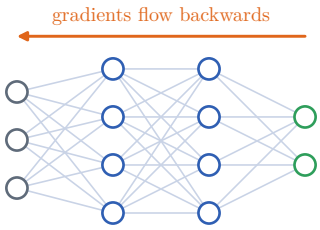
(e) 1955: the Dartmouth conjecture



(f) 1958: the Mark I Perceptron



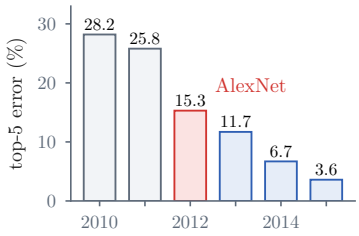
(g) 1959: checkers



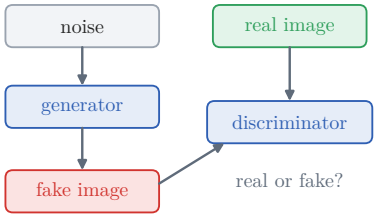
(h) 1986: a network trained by passing gradients backwards



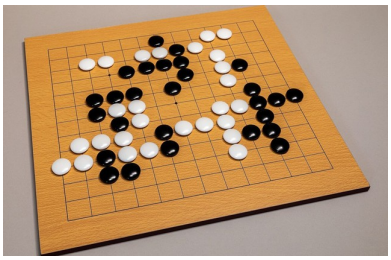
(i) 1997: Deep Blue



(j) 2012: the winning ImageNet error by year



(k) 2014: a generative adversarial network



(l) 2016: Go

Figure 2. Landmarks in computation and artificial intelligence. Photographs from Wikimedia Commons; the other panels were drawn for these notes.

5. **2023 and 2024: GPT-4 and GPT-4o.** Images as input, and then one model for text, audio, and images (Figure 3e).
6. **2024 and 2025: OpenAI o1 and DeepSeek-R1.** Reinforcement learning on problems with checkable answers taught models to reason at length before answering, spending more computation at inference time to get more answers right (Figure 3f).
7. **2025: gold-medal standard at the International Mathematical Olympiad.** An advanced version of Gemini Deep Think solved five of the six problems, for 35 of 42 points, graded by the competition's coordinators (Figure 3g).
8. **2025 and 2026: open Erdős problems.** Language models, sometimes paired with the Lean proof assistant, began resolving problems from Paul Erdős's collection; Problem 728, settled in January 2026 with a Lean-checked proof, is one example. Terence Tao's tracker records these contributions,

from literature search to complete solutions, and cautions that some of the problems had simply been overlooked.

9. **2026: the unit distance conjecture disproved.** Erdős conjectured in 1946 that n points in the plane determine at most $n^{1+o(1)}$ pairs at distance exactly one. In May 2026 an internal OpenAI reasoning model constructed configurations with at least $n^{1+\delta}$ such pairs, for a fixed $\delta > 0$, using ideas from algebraic number theory. Noga Alon, Tim Gowers, and other mathematicians checked the construction and wrote a companion paper (Figure 3h).
10. **2026, claimed: a Millennium Prize problem.** In September 2026 OpenAI announced that about 10,000 coordinated agents, running an internal model, had shown that a fluid initially at rest and driven by a smooth external force can develop a singularity in finite time under the three-dimensional Navier–Stokes equations, one of the outcomes the Clay problem allows. The proof was checked in Lean. The Clay Mathematics Institute said that the problem "has apparently been settled" and began a deliberately unhurried review. A priority dispute with concurrent human work, and whether a forced example answers the question mathematicians care most about, are still debated (Quanta) (Figure 3i).

Across these landmarks, language became a general interface for specifying tasks, models learned to follow instructions, their inputs widened beyond text, and their performance rose fastest on problems whose answers can be checked by a test or a proof checker. The last three items also show why a claim should be read together with its evidence: a formal proof, checking by experts, or a review still under way.

2. What we will study and build

The central object

A generative model, defined in §3, produces structured objects: a paragraph, a program, an image, a sequence of actions. This course concentrates on language, for two reasons. Language models are the systems that changed the field, and language is the setting in which the mathematics is richest: it has discrete symbols, recursive structure, compositional meaning, dependence on context, and a long history in logic, linguistics, and the theory of computation.

The object at the center of the course is a probability distribution over strings. Fix a finite vocabulary $\mathcal{V}$ of tokens (characters, words, or pieces of words) together with a special end-of-sequence token EOS. A language model assigns a probability to every finite string $x_1 x_2 \cdots x_T$ by predicting one token at a time:

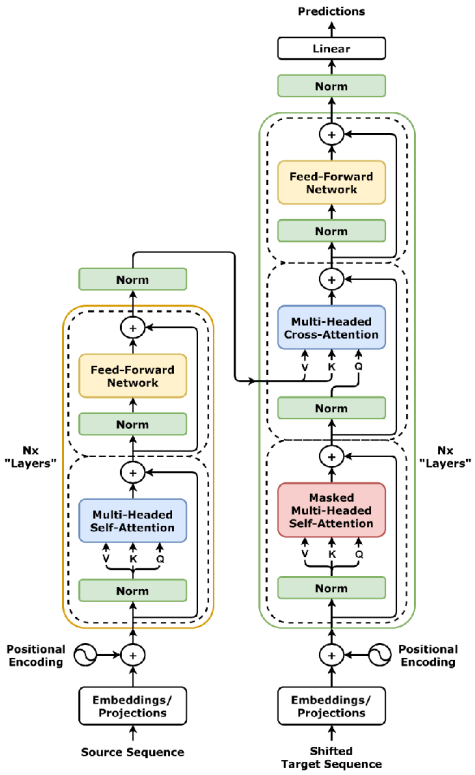
$$q_\theta(x_{1:T}, \text{EOS}) = \prod_{t=1}^{T+1} q_\theta(x_t \mid x_{<t}), \qquad x_{T+1} = \text{EOS}.$$

Here $x_{<t}$ denotes the tokens before position t, and θ collects the model's parameters.

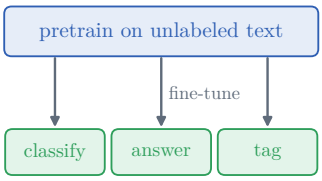
This way of modeling language is not new. The product is the chain rule of probability, and Claude Shannon generated English text one letter or one word at a time from such conditionals in 1948 (§6). What is new is how the conditionals $q_\theta(\cdot \mid x_{<t})$ are computed and fitted: by a neural network that maps the whole prefix to a distribution over the next token, with the Transformer architecture, trained on a large fraction of the written record with enormous amounts of computation, and then post-trained to behave as an assistant.

Almost everything in the course is a statement about this object:

- how the conditionals are computed: with counts, then with neural networks up to the LSTM (Week 2), then with attention (Week 3) and the Transformer (Week 4);
- how θ is fitted at scale, and what the fitted model can already do (Week 5);
- how the fitted distribution is changed so that sampling from it gives a helpful assistant rather than a plausible continuation of web text: the reinforcement learning this needs (Week 6), then post-training (Week 7);
- how the distribution is used: decoded, searched, served, and evaluated (Week 8).



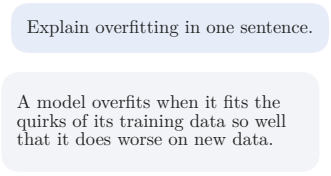
(a) 2017: the Transformer, an encoder (left) and a decoder (right)



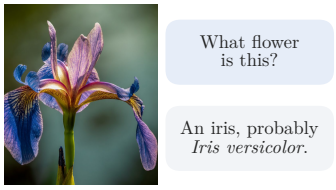
(b) 2018: pretrain, then fine-tune

```
Review: The plot dragged.
Sentiment: negative
Review: A warm, funny film.
Sentiment: positive
Review: I'd watch it again.
Sentiment: positive
```

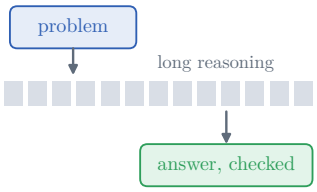
(c) 2020: a task from examples



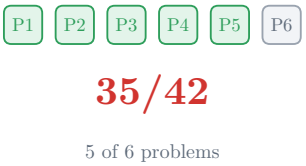
(d) 2022: an assistant



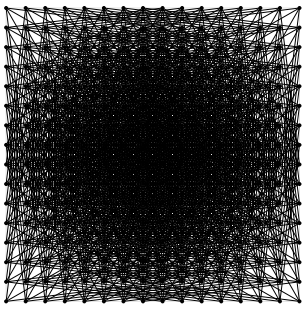
(e) 2023: an image as input



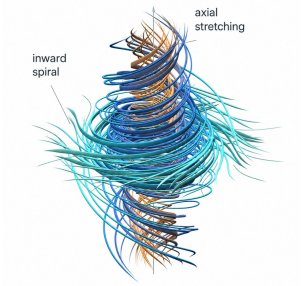
(f) 2024: reasoning before answering



(g) 2025: olympiad problems



(h) 2026: unit distances in a rescaled square grid, the construction the new result beats



(i) 2026: a swirling flow, spiraling inward and stretching along its axis

Figure 3. Landmarks in language modeling. Panel (a) is from Wikimedia Commons, and panels (h) and (i) are OpenAI's pictures from its announcements of the two 2026 results; the other panels were drawn for these notes.

What we will build

The course has two aims.

- **A mathematical understanding.** For each component: how text becomes vectors (tokens and embeddings); how a network turns a prefix into next-token probabilities (the Transformer); which objective training minimizes and what minimizing it achieves (the log loss, and later objectives built from preferences and rewards); what each step costs in computation and memory; and which claims about these systems are theorems, which are measurements, and which are conjectures.
- **A working understanding.** Implementing each component, training a model, fine-tuning it, and measuring what it does.

The second aim is served by one small codebase, `tinylm`, which we build from scratch:

- It gains one component each week: an automatic-differentiation engine (this lecture, §8), count-based language models, attention inside a trained translator, a Transformer, pretraining, policy gradients

on a small problem, a chatbot made by supervised fine-tuning, post-training with preferences and rewards, and fast generation and evaluation.

- The count-based models are baselines, not an early Transformer. What the Transformer replaces is the computation of the conditional probabilities.
- Every model trains on a laptop in minutes to an hour. The output is modest, but every mechanism of the large systems is present and can be inspected and tested.
- *From scratch* means that we write the models and their training ourselves, from tensor operations, and the differentiation engine itself once, in §8. From Week 2 on we use PyTorch's tensors and automatic differentiation, and no higher-level model libraries.

The eight weeks

Week	Lecture	Central development	We build
1	(Artificial) Intelligence, Learning, Language, and Meaning	History and vocabulary; intelligence, learning, language, and meaning	A tensor autograd engine; logistic regression and a small network
2	Language Modeling: From Counts to Neural Networks	n-gram models and perplexity; word embeddings, from counts and from word2vec; neural and recurrent language models	Count-based language models and an evaluation harness, word2vec, a neural language model, and character-level RNN and LSTM models
3	Sequence Modeling, Translation, and Attention	Sequence transduction; translation and word alignment; recurrent encoder–decoders; attention and its variants; convolutional sequence models	The attention step of a trained translator, checked against the model
4	Transformers: Architecture, Computation, and Language Modeling	Self-attention, masks, and heads; the Transformer block; the decoder-only language model; parameters, computation, and the cache	A decoder-only Transformer, tested for causality
5	Pretraining and Base Models: Data, Scale, and In-Context Learning	Tokenization, data, optimization, and scaling laws; what a base model can do: prompting and in-context learning	A BPE tokenizer, a training loop, and models of several sizes on one budget
6	Reinforcement Learning: Returns, Policy Gradients, and Policy Improvement	Returns and values; the policy gradient, baselines, and critics; controlled policy updates and PPO	REINFORCE with and without a baseline
7	Post-Training: Demonstrations, Preferences, and Rewards	Supervised fine-tuning; preferences and reward models; KL-regularized optimization, DPO, and verifiable rewards	A chatbot by supervised fine-tuning; DPO; GRPO on arithmetic as an extension
8	Inference: Decoding, Test-Time Compute, Serving, and Evaluation	Decoding, test-time computation, serving, and evaluation	Samplers, a key–value cache, and speculative decoding
9*	Alignment and Language Model Agents	Specifying and evaluating desired behavior; tools, retrieval, and agent loops	A tool-use loop

*Week 9 is optional.

The order follows the dependencies between ideas. Count-based language modeling comes first because its questions (sparsity, evaluation, what a distribution over strings is) remain the questions of the neural era. Attention comes before the Transformer, and the Transformer before scale, because attention is the Transformer's central operation and a scaling law is a statement about a fixed architecture. Reinforcement learning gets a lecture of its own before post-training, because the post-training objectives are much easier to understand once the underlying decision problem is clear.

The point of view

The course asks the same four questions of every system it studies:

- **What is computed?** A trained Transformer computes a specific function; decoding computes another; a chat system wraps both in a larger program. Being precise about the function is the first step to reasoning about it.
- **With what resources?** Training compute, memory, the cost of one forward pass, and the cost of generating a complete answer are different quantities, and they scale differently.
- **From what information?** A learner can only extract what its data contain. Much of the course is about what each stage of training supplies: raw text, demonstrations, comparisons, or checkable outcomes.
- **With what guarantee?** Some claims about language models are theorems, some are measurements, and some are conjectures supported by a handful of experiments.

A recurring case is the separation of **expressivity**, **optimization**, and **generalization**. A model class may be able to express a computation without any efficient procedure finding it, and a procedure may fit the training data without the result working on new inputs. Much confusion about what language models "can do" comes from running these three questions together.

3. Terminology

The opening used words such as intelligence, learning, and model freely. This section fixes their meanings. Foundations 1 of the Background Notes on Machine Learning and AI develops the same vocabulary with more examples, and §9 lists where each basic topic is treated in full.

AI, machine learning, and deep learning

Artificial intelligence is the study of computational systems that perform tasks associated with intelligence: perception, reasoning, language, planning, and action. Its methods include search (exploring a space of possibilities, as in game playing and planning), logic (representing knowledge and deriving consequences), probabilistic reasoning (representing uncertainty with probability distributions and updating them with evidence), and learning. A practical system usually combines several of these: a chat assistant is a learned model inside a program that manages the conversation, may search over candidate answers, and may call tools.

Machine learning is the study of procedures that improve with experience. Tom Mitchell's definition is still the most precise short one: a program learns from experience E with respect to a class of tasks T and a performance measure P if its performance on tasks in T, as measured by P, improves with E. For a language model, T might be predicting the next token of web text, E a corpus of documents, and P the average log probability assigned to held-out text.

Deep learning is machine learning with models built as compositions of many trainable transformations, usually neural networks, so that the model learns intermediate representations of its input along with its predictions. The depth is the length of the composition.

Sources of learning signal

Learning procedures differ in where their training signal comes from.

- **Supervised learning** uses examples of inputs paired with targets, (x_i, y_i): images with labels, or instructions with demonstrated answers.
- **Unsupervised learning** receives inputs without targets and models their structure: a density, clusters, or a low-dimensional representation.
- **Self-supervised learning** constructs targets from the data itself. A language model predicts each token from the ones before it; a masked model predicts hidden tokens from the visible ones. The provenance of the target differs from supervised learning, but the training calculation is an ordinary supervised loss.
- **Reinforcement learning** learns from rewards obtained by acting. The learner's actions affect what it observes next, and rewards may arrive long after the actions that earned them.

These categories describe the source of the signal, not the architecture. A single Transformer is pretrained with self-supervision, fine-tuned with supervision, and adjusted with rewards, as the course will describe week by week.

Models, representations, and architectures

- A **model** is a mathematical object that represents a relationship or a distribution, such as the spam probability $q_w(1 \mid x)$ below. A **model class** is a family of models indexed by parameters.
- A **representation** is the form in which an input reaches the model: the features or vectors it receives. It decides what information the model can use. A bag of words, which counts how often each word occurs, gives "dog bites man" and "man bites dog" the same representation; a sequence of tokens keeps them apart. Representations can also be learned: each hidden layer of a network computes a new representation of the input (Foundations 1, representation learning).
- An **architecture** is the form of the computation from representation to output: which operations are applied, in what order, and which parameters are shared between them. It fixes the model class, and learning then chooses a member of the class. Logistic regression is a linear function followed by a sigmoid; a convolutional network applies the same small filter at every position of an image; a Transformer applies the same parameters at every position of a sequence (Foundations 1, neural networks).
- **Parameters** are the numbers that learning adjusts, such as the weights of a network. **Hyperparameters** are choices made outside learning, such as the learning rate, the width of a network, or a regularization strength; they are set during development.
- A **learning algorithm** maps data to parameters. A separate computation then evaluates the fitted model on new inputs.

The architecture also shapes learning itself. Gradient descent (§5) sends the learning signal backwards along the architecture's own computation graph, since backpropagation runs that graph in reverse (§8). The architecture therefore decides which parameter settings gradient descent reaches easily, not only which ones exist. Residual connections in deep networks, for example, are there largely so that gradients pass through many layers without fading.

For example, represent an email by a vector x of features, with label $y = 1$ for spam and $y = 0$ otherwise. Logistic regression models the conditional probability of spam as

$$q_w(1 \mid x) = \sigma(w^\top x), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The vector w holds the parameters, $w^\top x$ is called the **logit**, and a decision rule such as "flag as spam if $q_w(1 \mid x) > 1/2$" turns the probability into an action.

Generative and discriminative models

A **generative model** is a probability distribution q_θ over the objects themselves, such as images, strings, or sequences of actions, fitted to data so that it is close to the unknown distribution p that produced them. Generating means sampling $x \sim q_\theta$. A **conditional** generative model $q_\theta(x \mid c)$ does the same given a context c, such as a prompt or a class label. A language model is a generative model of text: a distribution over strings, sampled one token at a time (§2).

In classification the word has an older, narrower sense. A **generative** classifier models the joint distribution of inputs and labels, often as $q(x, y) = q(x \mid y)q(y)$, and classifies with Bayes' rule,

$$q(y \mid x) = \frac{q(x \mid y)\, q(y)}{\sum_{y'} q(x \mid y')\, q(y')}.$$

A **discriminative** classifier models $q(y \mid x)$ directly, as logistic regression does; Ng and Jordan (2001) compare the two. Both can classify, but only the generative classifier can also produce new inputs, by sampling a label and then an input of that class. A language model is generative even though each of its conditionals $q_\theta(x_t \mid x_{<t})$ is a classifier over the next token.

Training, generalization, and evaluation

A **loss** measures the cost of a prediction. For probabilistic predictions the standard choice is the log loss $\ell(\theta; x, y) = -\log q_\theta(y \mid x)$, which penalizes assigning low probability to what actually happened. If

examples are drawn from a population distribution p_*, the quantity we care about is the **population risk** $R(\theta) = \mathbb{E}_{(x,y)\sim p_*}[\ell(\theta; x, y)]$. We can only compute the **empirical risk** on the n training examples,

$$\widehat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; x_i, y_i),$$

and **empirical risk minimization** chooses θ to make it small, often with a regularization term added.

Generalization is the relationship between the two. **Overfitting** occurs when the empirical risk falls without a corresponding fall in the population risk (Figure 4). The Occam bound of §5 is the simplest example of a generalization guarantee: with enough examples relative to the size of the hypothesis class, empirical and population performance are close.

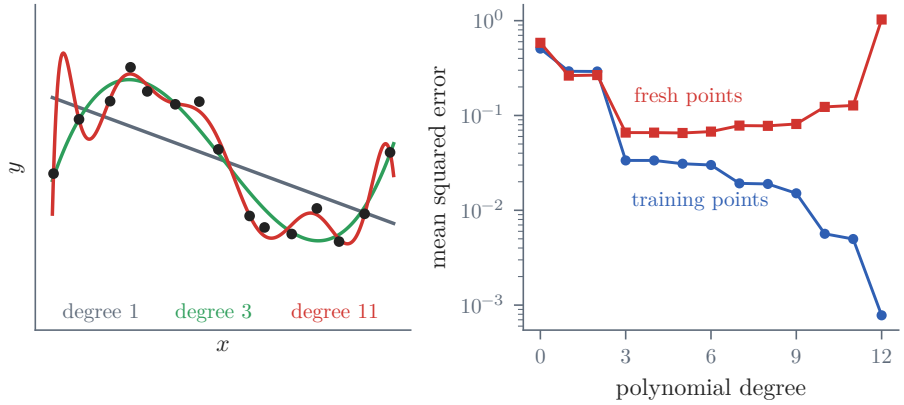


Figure 4. Overfitting. Left: polynomials of degree 1, 3, and 11 fitted to the same 15 noisy points from a sine curve. Right: as the degree grows, the error on the training points keeps falling, while the error on fresh points falls and then rises.

In practice, generalization is estimated rather than proved. Training data fit the parameters; **validation data** guide development choices such as the architecture, learning rate, and stopping time; **test data** estimate performance once those choices are fixed. The separation concerns information flow, not filenames: a test set that has been used to choose a model has become part of development. For text, overlapping windows or duplicated passages can leak information across a split even when the split looks clean, so splits are made by document, and duplicates are removed.

Training changes a model's parameters; running the trained model with its parameters fixed is called **inference**. For language models, training itself has stages: **pretraining** on a large corpus of text, and **post-training**, which adapts the pretrained model with demonstrations, comparisons, or rewards. §8 summarizes the stages and the signal each one uses.

The word **inference** is badly overloaded, and it is worth separating its meanings once:

Term	Meaning	Example
Statistical inference	Reasoning from data to unknown quantities, including their uncertainty	A confidence interval for a model's accuracy
Probabilistic inference	Computing a quantity within a specified model	The posterior probability of spam given the words of an email
Inference (deployment)	Running a trained model	Generating a response to a prompt

4. Intelligence

This section follows the question of machine intelligence in order: how it was posed and turned into a test, how intelligence has since been defined and measured, how the field set out to build it, how two celebrated systems succeeded, and the lesson that the history of such systems suggests.

Turing's question

The question "Can machines think?" is older than computers. In 1637 René Descartes argued, in Part V of his *Discourse on the Method*, that a machine could never arrange words to reply sensibly to whatever

is said in its presence, as even the dullest person can. In 1843 Ada Lovelace wrote that Charles Babbage's Analytical Engine "has no pretensions to originate anything": it can do only what it is ordered to do.

By 1949 the question had reached the newspapers. Stored-program computers were running in Manchester and Cambridge, and the neurosurgeon Geoffrey Jefferson argued in a widely reported lecture that no machine would equal the brain "until a machine can write a sonnet or compose a concerto because of thoughts and emotions felt, and not by the chance fall of symbols." Alan Turing, then working on the Manchester computer, answered both Jefferson and Lovelace in *Computing Machinery and Intelligence* (1950).

Turing judged "Can machines think?" too ill-defined to discuss and replaced it with a game. An interrogator converses by text with two hidden parties, a person and a machine, and must decide which is which. The machine does well if the interrogator is wrong about as often as with two people. Turing predicted that by about the year 2000 a machine with roughly 10^9 bits of storage would play well enough that an average interrogator would have no more than a 70 percent chance of a correct identification after five minutes.

Two features of the paper matter for this course. First, the test is behavioral and linguistic: intelligence is to be judged by conversation, which is exactly the capability that language models now exhibit. Second, most of the paper is not about the game at all but about objections to machine intelligence, and its final section, "Learning Machines," argues that the practical route to a thinking machine is not to program adult competence directly: "Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child's?" The child machine would then be educated. Against Lovelace, Turing observes that a learning machine's behavior is not fully anticipated by its programmer. Seventy years later, the systems that come closest to passing his test were built in roughly the way he suggested: a general learning procedure, a great deal of experience, and comparatively little explicit programming.

The game's lasting value is methodological: it replaces a metaphysical question with an operational one about observable behavior. Much of this course does the same at a smaller scale, replacing "does the model understand?" with "what does the model compute, and how does it perform on a specified distribution of tasks?"

As a definition of intelligence, however, the imitation game is poor, and Turing did not offer it as one. It is anthropocentric, it rewards deception, and people attribute understanding on thin evidence. Joseph Weizenbaum's 1966 program ELIZA, a few pattern-matching rules that turned a user's statements back into questions, left some of its users convinced that it understood them.

ELIZA shows that behavior can suggest more understanding than there is. John Searle's Chinese room (1980) attacks the behavioral view from the other side: even flawless behavior, he argues, would not establish understanding. A person who speaks no Chinese sits in a room with a rulebook for manipulating Chinese characters. Questions in Chinese come in; by following the rules, the person sends out answers that native speakers find perfectly sensible. Searle argues that the person does not understand Chinese, that neither does the room, and therefore that running the right program is not sufficient for understanding. The most common reply is that the system as a whole, not the person, is the relevant candidate for understanding.

Whether the argument applies to language models is itself disputed. A language model resembles the room in being a fixed procedure that maps input strings to output strings, with its competence derived from text rather than from contact with the world. It differs in that its procedure was learned rather than written, and in that no single component plays the part of the person. The analogy settles nothing, but it sharpens a question that §7 takes up: can meaning be learned from form alone?

Definitions and measures

Turing sidestepped the question of what intelligence is. There is no agreed definition, but there are useful families of definitions. The standard textbook of the field, Stuart Russell and Peter Norvig's *Artificial Intelligence: A Modern Approach*, sorts them along two axes: whether they concern thinking or acting, and whether they measure against human performance or against an ideal of rationality. The book adopts the rational-agent view: an agent perceives its environment and acts so as to maximize the expected value of a performance measure, given what it has perceived. This view turns intelligence into an optimization problem, and it is the view behind the reinforcement learning of Week 6.

Shane Legg and Marcus Hutter pushed the rational-agent view to a formal limit. The universal intelligence of an agent is its expected performance across all computable environments, each weighted by its simplicity, so that simple environments count most. The definition captures a common intuition,

that intelligence is the ability to do well across many environments with a bias toward simple ones. It is also incomputable, so it cannot serve as a benchmark; §A.1 gives the formula.

François Chollet objects that measuring skill at tasks, even many tasks, conflates intelligence with the knowledge and priors used to build the skill: a system can buy performance on any fixed set of tasks with enough training data. He proposes to measure intelligence as *skill-acquisition efficiency*: how quickly a system acquires new skills over a scope of tasks, relative to its priors, its experience, and the difficulty of generalization. The distinction matters for language models, which are trained on a large fraction of the written record. Their performance on a benchmark says something about their skill, but by itself very little about how efficiently they learn.

Symbols, networks, and the Dartmouth conjecture

Definitions aside, the field that set out to build intelligent machines began with a conjecture. The 1955 proposal for the Dartmouth summer workshop, by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon, named the field and stated its founding conjecture: "every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it." Its list of topics included how a computer can be programmed to use a language, neuron nets, self-improvement, abstraction, and randomness and creativity. Almost every item on that list is a topic of this course.

Two broad programs developed from it.

- **The symbolic program** held that intelligence consists in manipulating symbols according to rules. Allen Newell and Herbert Simon stated its strongest form as the physical symbol system hypothesis: "a physical symbol system has the necessary and sufficient means for general intelligent action." Theorem provers, planners, expert systems, and hand-written grammars belong to this program.
- **The connectionist program** held that intelligence emerges from large numbers of simple units adjusting their connections. It begins with Warren McCulloch and Walter Pitts's 1943 logical model of neurons, which showed that networks of threshold units can compute any Boolean function, and with Frank Rosenblatt's perceptron, a single threshold unit that learns its weights from labeled examples. A short film clip from the period shows the Mark I Perceptron at work. Marvin Minsky and Seymour Papert's 1969 book *Perceptrons* proved that single-layer perceptrons cannot compute predicates such as parity or connectedness from local features, and the lack of an effective training method for multilayer networks stalled the program until backpropagation was popularized in the 1980s.

A modern language model descends from the connectionist program: a network of simple units whose connections are learned. Yet it operates on discrete symbols, produces symbol sequences, and can be prompted to carry out symbolic procedures such as arithmetic or program execution, sometimes reliably and sometimes not. Whether it manipulates symbols in Newell and Simon's sense, and whether that question has a sharp answer, is one of the live debates this course equips you to think about.

Search and learning: Deep Blue and AlphaGo

The two programs differ most visibly in where a system's knowledge comes from: written down by people, or learned from data. Game playing makes the contrast sharp, because success in it is unambiguous, and two landmarks show how differently it can be achieved.

IBM's Deep Blue, which defeated the world chess champion Garry Kasparov in a six-game match in 1997, combined a deep search of the game tree with an evaluation function built by hand from thousands of features of a chess position. It ran on 480 custom chips that together examined on the order of a hundred million positions per second. Its knowledge of chess came from people; its strength came from search.

Go resisted this approach. A position has roughly 250 legal moves rather than about 35, and nobody could write a good evaluation function by hand. DeepMind's AlphaGo, which defeated Lee Sedol 4–1 in 2016, learned the functions it needed. A policy network, trained first to predict the moves of expert players and then improved by playing against itself, proposed promising moves; a value network, trained on positions from self-play, estimated who was winning; and a tree search combined the two.

Both systems search. They differ in where their knowledge comes from, and the same division runs through this course. A pretrained language model is a learned policy over next tokens. Post-training with rewards (Week 7) improves it much as self-play improved AlphaGo's policy, and the test-time

computation of Week 8, which samples many answers, verifies them, or searches over steps of reasoning, is search wrapped around a learned model.

The bitter lesson

Deep Blue and AlphaGo are two points on a longer trend. In a short essay Richard Sutton observed that across seventy years of AI research, general methods that exploit increasing computation, chiefly search and learning, have eventually beaten methods that build in human knowledge about the domain, often by a large margin. The progression from hand-written grammars to n-gram models to neural language models trained on trillions of tokens is one of the clearest instances. The lesson is empirical, not a theorem, and it does not say that structure is useless: the Transformer is itself a carefully designed structure, and much of Weeks 3 and 4 is about which structural choices matter at scale.

In a 2025 conversation with Dwarkesh Patel (Figure 5), Sutton argued that language models do not yet follow his own lesson: they learn from text that humans wrote rather than from their own experience of acting in the world, and he expects systems that learn continually from experience to overtake them. The disagreement is a useful one to keep in mind through Weeks 6 and 7, where language models do learn from rewards for their own outputs.



Figure 5. Richard Sutton on the Dwarkesh Podcast, 2025.
Source: thumbnail of the video of the episode.

5. Learning

This section builds up learning in the order the course needs it: why learning from examples needs assumptions, what can then be learned and at what cost, the simplest learning machine and how it grew into deep networks, how networks are fitted, and what the log loss that fits them means.

The problem of induction

Every learning system generalizes from what it has seen to what it has not. David Hume observed in 1748, in his *Enquiry Concerning Human Understanding*, that no argument from experience can establish that unobserved cases will resemble observed ones without assuming the very regularity it sets out to prove. Any justification of induction must therefore rest on assumptions about the world that the data alone cannot supply.

Machine learning has a precise version of this observation, usually called a no-free-lunch theorem. Consider binary classification on a finite domain of N points, and suppose the true labeling is chosen uniformly at random from all 2^N functions. After observing the labels of any subset of the points, the label of every unobserved point is still a fair coin, independent of everything seen. Every learning algorithm, however clever, then has expected error exactly one half on unseen points.

Learning is possible only because real problems are not like this: the functions we care about are smooth, compositional, or otherwise simple, and a learner succeeds by preferring such functions. That preference is its **inductive bias**. In a language model the inductive bias lives in the tokenizer, the architecture, the optimizer, the order of the data, and the objective, and much of the course is about what each of these contributes.

Learning as a computational problem

The no-free-lunch theorem says that learning needs assumptions. Leslie Valiant's model says which ones, and what learning then costs: in *A Theory of the Learnable* (1984) he turned learning into a problem about algorithms and resources. In the probably approximately correct (PAC) model, there is an unknown distribution $\mathcal{D}$ over an input domain X and an unknown target concept $c : X \to \{0,1\}$ drawn from a known class $\mathcal{C}$. The learner receives m independent examples $(x_i, c(x_i))$ with $x_i \sim \mathcal{D}$ and must output a hypothesis h whose error

$$\mathrm{err}_{\mathcal{D}}(h) = \Pr_{x \sim \mathcal{D}}[h(x) \neq c(x)]$$

is at most ε, with probability at least $1 - \delta$ over the sample. The class is efficiently PAC learnable if some algorithm does this for every $\mathcal{D}$ and every $c \in \mathcal{C}$, using a number of examples and an amount of time polynomial in $1/\varepsilon$, $1/\delta$, and the size of the problem.

The definition makes both halves of Hume's problem explicit. The distribution $\mathcal{D}$ is the assumption that the future resembles the past: test inputs come from the same distribution as training inputs. The class $\mathcal{C}$ is the inductive bias: the learner knows the target is one of a restricted set of functions. Given these, learning becomes possible, and its cost can be measured.

The simplest positive result shows how. If the learner searches a finite hypothesis class $\mathcal{H} \supseteq \mathcal{C}$ and outputs any hypothesis that agrees with every training example, then

$$m \;\geq\; \frac{1}{\varepsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$$

examples suffice. This is the **Occam bound**, and its proof takes a few lines (§A.2). The number of examples grows with $\ln|\mathcal{H}|$, the number of bits needed to name a hypothesis. For the class of all Boolean functions on n bits the bound is exponential in n: the no-free-lunch phenomenon reappears as a statement about sample size.

The Occam bound says nothing about computation time, and time can be the binding constraint. Under standard cryptographic assumptions, deterministic finite automata cannot be learned efficiently from random examples. Yet Dana Angluin's L* algorithm learns them exactly in polynomial time if the learner may ask whether particular strings belong to the language and may propose an automaton to be corrected with a counterexample (§A.3). What a learner can learn efficiently depends on the channel through which it gets information. The theme recurs throughout the course: pretraining on text supplies random examples, supervised fine-tuning supplies chosen demonstrations, preference learning supplies comparisons, and verifiable rewards supply something like answers to queries.

From the perceptron to deep learning

The PAC model says what can be learned in principle. The learning systems of this course descend from one simple, concrete procedure. Rosenblatt's perceptron classifies an input $x \in \mathbb{R}^d$ by the sign of $w^\top x$, and it learns from its mistakes: whenever it misclassifies an example (x, y) with label $y \in \{-1, +1\}$, it updates $w \leftarrow w + yx$. If some unit vector u separates the data with margin γ, meaning $y\,u^\top x \geq \gamma$ for every example, and every input has norm at most R, then the perceptron makes at most $(R/\gamma)^2$ mistakes, however many examples it sees (Novikoff, 1962; Chapter 9 of Shalev-Shwartz and Ben-David). The bound depends on the geometry of the data, not on the dimension or the number of examples. Like the Occam bound, it is a guarantee that holds whatever the data, given one assumption about them, here a margin.

A single threshold unit can only draw a linear boundary, the limitation Minsky and Papert made precise. The simplest pattern it cannot learn is exclusive or (Figure 6, left): the points $(0,1)$ and $(1,0)$ are positive, $(0,0)$ and $(1,1)$ are negative, and no line separates them.

Stacking units removes the limitation. In Figure 6 two perceptrons each draw one line, and a third unit takes their two outputs as its inputs and weighs them: it labels a point positive when the point lies on the positive side of both lines. The output unit is a weighted vote over the opinions of the units below it, a smoothed majority vote in which the network also learns how much to trust each voter.

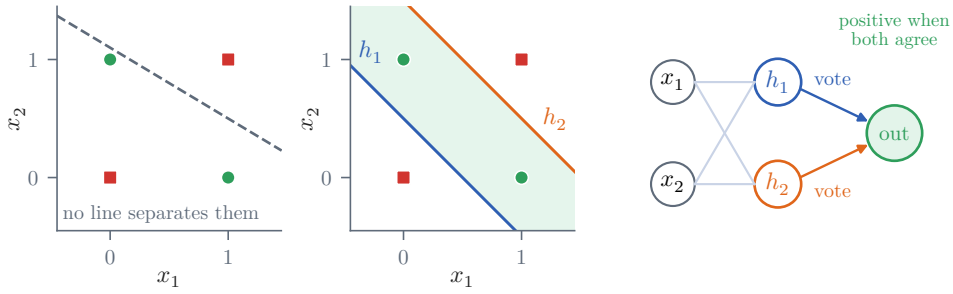


Figure 6. Exclusive or. Left: no single line separates the two classes. Middle: two hidden units each draw a line. Right: the output unit combines their votes and labels the band between the lines positive.

The perceptron's rule cannot train such a network, however. When the output is wrong, the rule would need to know which hidden unit to blame, and in which direction to move it, but a hidden unit has no target of its own. The answer, popularized by Rumelhart, Hinton, and Williams in 1986, is to make every unit differentiable, replacing the hard threshold with a smooth function such as the sigmoid, and to follow the gradient of a loss. The chain rule then assigns each hidden unit its share of the blame. Welch Labs' video *Why Deep Learning Works Unreasonably Well* shows how stacking such units lets a network represent increasingly complicated functions.

Neural networks and gradient descent

A feedforward neural network composes affine maps with fixed nonlinear functions: a layer computes $h \mapsto \phi(Wh+b)$, where W and b are parameters and ϕ is applied coordinatewise, for example $\phi(z) = \max(0, z)$. Without the nonlinearity a stack of layers would collapse to a single affine map. With it, networks with one hidden layer of sufficient width can approximate any continuous function on a compact set to any accuracy (Cybenko, 1989; Hornik, 1991).

The universal approximation theorem is a statement about expressivity only. It says nothing about how large the network must be, how many examples are needed to find a good approximation, or whether gradient descent will find it. Learning parity functions of a few hidden bits from random examples, for instance, is easy to express and notoriously hard for gradient methods. The success of deep learning is an empirical fact about how expressivity, optimization, and generalization interact on natural data, and the theory explaining it is incomplete (LeCun, Bengio, and Hinton, 2015).

Architectures differ in how they share parameters, which encodes an assumption about the data. A convolutional network reuses the same filters at every position of an image, assuming that local patterns matter wherever they occur. A recurrent network applies the same update at every position of a sequence, carrying a hidden state that summarizes the past. A Transformer lets every position of a sequence read from every earlier position through attention, with the same parameters at every position. Weeks 2 and 3 develop the last two in detail.

Networks, like most models in this course, are fitted by gradient methods. **Gradient descent** repeatedly moves the parameters against the gradient of the loss, $\theta \leftarrow \theta - \eta \nabla \widehat{R}(\theta)$, with a learning rate $\eta > 0$. **Stochastic gradient descent** replaces the full gradient with an unbiased estimate computed from a small random subset of examples, a **minibatch**, trading noise for much cheaper steps. Variants such as momentum, Adam, and AdamW adapt the step using the history of gradients; Week 5 examines them.

For logistic regression with log loss, the gradient on one example is $(\sigma(w^\top x) - y)\, x$: the prediction error times the input. Starting from $w = (0, 0)$ with $x = (2, 1)$ and $y = 1$, the predicted probability is $1/2$, the loss is $\ln 2 \approx 0.693$, and the gradient is $-\frac{1}{2}(2, 1)$. One step with $\eta = 0.1$ gives $w = (0.1, 0.05)$, and the predicted probability of the correct label rises to $\sigma(0.25) \approx 0.562$. The step improved the prediction on the example it was computed from. Whether it helps on other examples is a separate question, which only held-out data can answer: on a second example $x' = (-2, 1)$ with $y' = 1$, the same step raises the loss from 0.693 to 0.771.

The gradient itself is computed by **backpropagation**, the chain rule applied backwards through the network. §8 builds an engine that does this for any computation made of differentiable operations.

Deep learning, the training of many such layers end to end on large datasets, displaced engineered features first in speech and image recognition, most visibly when AlexNet won the 2012 ImageNet competition by a wide margin, and then in language.

Prediction is compression

The networks of this course output probabilities and are trained by minimizing the log loss (§3), the negative log probability of what actually happened. The log loss has a second meaning, which ties learning to information theory: it is a code length. A symbol predicted with probability $1/2$ can be encoded in one bit; a symbol predicted with probability $1/64$ costs six. In general, given any probability distribution q over messages, there is a prefix code, arithmetic coding, that encodes a message x in at most $-\log_2 q(x) + 2$ bits. Conversely, by Kraft's inequality, the code lengths $\ell(x)$ of any prefix code define weights $2^{-\ell(x)}$ that sum to at most one, so every code is implicitly a probability model.

If messages are drawn from a source p, the expected code length when coding with q is essentially the cross-entropy

$$H(p, q) = -\sum_x p(x) \log_2 q(x) = H(p) + D_{\mathrm{KL}}(p \,\|\, q),$$

where $H(p)$ is the entropy of the source and D_{KL} is the Kullback–Leibler divergence, which is nonnegative and zero only when $q = p$. A model that predicts better compresses better, and the gap to the best possible compression is exactly the KL divergence.

For independent examples, maximizing the likelihood of the data is the same as minimizing the total log loss. For a model $q_\theta(x)$ fitted to samples from p_*, the expected log loss is the cross-entropy, so

$$\mathbb{E}_{x \sim p_*}[-\log q_\theta(x)] = H(p_*) + D_{\mathrm{KL}}(p_* \,\|\, q_\theta).$$

The entropy $H(p_*)$ does not depend on θ, so in the limit of infinite data maximum likelihood minimizes the KL divergence from the true distribution to the model. This one identity connects the language-model training objective to compression and to the perplexity evaluations of Week 2.

Training a language model by minimizing next-token log loss is, in this sense, training a compressor for text. Grant Sanderson's video *But what is cross-entropy?*, the second part of his series *Compression is Intelligence*, develops this correspondence visually. It also runs the other way: every compressor is implicitly a predictor. Nathan Barry's GziPT generates Shakespeare-like text with no neural network at all, by choosing at each step the continuation that gzip compresses best.

The same correspondence explains the old preference for simple hypotheses. Naming a hypothesis costs bits too: the Occam bound above charges $\ln|\mathcal{H}|$, the length of a name in a class of $|\mathcal{H}|$ hypotheses. The minimum description length principle makes the trade-off explicit: prefer the hypothesis that minimizes the length of its own description plus the length of the data coded with its help, so that a hypothesis earns its complexity only by compressing the data. Ray Solomonoff took the idea to its limit in 1964: weight every program that prints the data by a prior that halves with each bit of its length, and predict with the mixture. That predictor cannot be computed, but for any computable source its total prediction error is bounded by a constant that depends only on the length of the source's description (§A.4). Good prediction, short description, and simplicity are three views of one idea. Whether compression of the written record amounts to understanding is a question for §7.

6. Language: rules, statistics, and context

What a sentence is

A written sentence is at once a sequence of marks, a grammatical construction, an expression about something, and a move in an interaction. The history of thinking about language has treated these aspects separately, and the distinctions remain useful for thinking about what a language model does and does not capture. This section follows the first two, structure and statistics; §7 takes up meaning and use.

Rules and productivity

Pāṇini's *Aṣṭādhyāyī*, a grammar of Sanskrit usually dated to the middle of the first millennium BCE, is the oldest grammar of any language that survives complete. It was not the first: Pāṇini names ten earlier grammarians, whose works are lost. In nearly four thousand short rules, together with technical conventions for applying them, it derives the forms of the language.

Its central insight is **productivity**: a finite description can determine an unbounded number of expressions, including ones never observed. Noam Chomsky made the same point the foundation of

modern linguistics. In *Three Models for the Description of Language* (1956) he described a grammar as a finite device that generates exactly the grammatical sentences of a language, and classified such devices by their power in what became the Chomsky hierarchy (Figure 7). The course does not pursue the hierarchy further: the models we build take the statistical route described next.

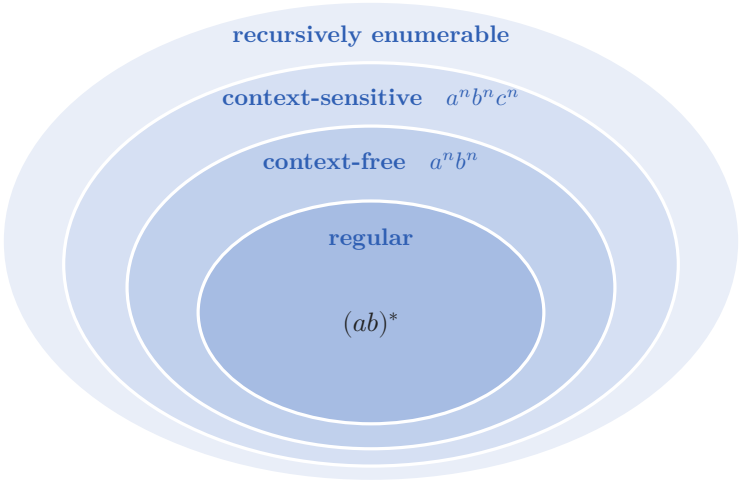


Figure 7. The Chomsky hierarchy. Each class of languages contains the ones inside it; the examples, strings of the form a...a b...b (c...c) with equal counts, need progressively more powerful grammars.

Learning a language from examples

If a language is given by a finite grammar, how can the grammar be learned? Children learn their first language from a finite sample of it, mostly without being told which sentences are ungrammatical. How this is possible was a defining question of twentieth-century linguistics, and it has recently become an active question in the theory of language models.

E. Mark Gold gave it a formal version in *Language Identification in the Limit* (1967). A learner knows that the language comes from some class of candidate languages. It is shown the strings of the language one at a time, in an arbitrary order, and after each string it guesses a grammar. It succeeds if, for every language in the class and every order of presentation, its guesses eventually settle on a correct grammar.

Gold's answer was negative. No learner succeeds on any class that contains all the finite languages and at least one infinite language, so not even the regular languages can be learned in this sense. The obstacle is overgeneralization: a learner that has seen only strings of a finite language cannot tell whether to stop there or generalize to a larger language that also contains them. If the learner is also told which strings are not in the language, much larger classes become learnable. Dana Angluin later characterized exactly which classes can be learned from positive examples alone (Angluin, 1980): each language must contain a finite set of "tell-tale" strings that no candidate language strictly inside it also contains.

The results were taken up in the debate over the **poverty of the stimulus**, Chomsky's argument that the evidence available to a child is too thin to determine a grammar without strong innate constraints on the possible grammars (Chomsky, 1959).

Gold's order of presentation is adversarial, however. When sentences are drawn at random from a probability distribution, the picture changes: James Horning showed in 1969 that probabilistic context-free grammars can be learned, in a probabilistic sense, from random positive examples. Frequencies carry information that a bare list of sentences does not. A construction that should appear often but never does is evidence that it is ungrammatical. Learning from random samples of text, with frequency as evidence, is exactly the setting of a language model.

A recent line of work asks for less than Gold did. A language model need not identify its language; it only has to produce new, valid strings of it. Jon Kleinberg and Sendhil Mullainathan showed in *Language Generation in the Limit* (2024) that this weaker goal is achievable for every countable class of languages, in sharp contrast with identification.

Their generator can succeed by retreating to a thin, safe part of the language, and the follow-up work turns this into a trade-off between validity and breadth, or in the vocabulary of language models, between hallucination and mode collapse:

- Kalavasis, Mehrotra, and Velegkas (2025) show that for a large class of generators, including next-token predictors, being eventually both valid and broad is impossible for most classes of candidate languages; with negative examples as well, which post-training can supply, both become possible.
- Kleinberg and Wei (2025) show that, even from positive examples alone, a generator can keep a positive fraction of the language, measured by density, in its outputs.

Prediction and statistics

The statistical view, in which frequencies are evidence, begins with Claude Shannon's 1948 paper founding information theory, *A Mathematical Theory of Communication*, which treated English as a stochastic source. It generated text from a series of approximations to English: letters chosen at random, then independently with their English frequencies, then letters chosen given the previous one or two, then words chosen given the previous word. The samples look more like English as the context grows.

To measure how predictable English is, Shannon asked people to guess the next letter of a text. He began with his wife, Betty Shannon, who guessed a book letter by letter while he recorded her guesses; Grant Sanderson tells the story in *Reinventing Entropy*, the first part of *Compression is Intelligence*. In *Prediction and Entropy of Printed English* (1951) Shannon estimated about one bit per letter when a long context is available, with bounds between 0.6 and 1.3 bits. Figure 8 plays the same game with counting models trained on two of Jane Austen's novels. The n-gram models of Week 2 are Shannon's approximations, and next-token prediction is his guessing game played by a machine.

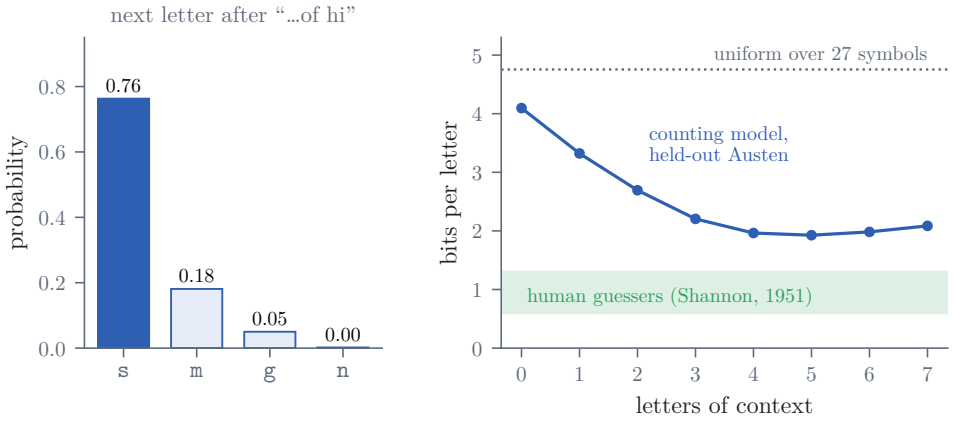


Figure 8. Shannon's guessing game. Left: a counting model's probabilities for the letter after "she could not help thinking of hi". Right: bits per letter on held-out text as the counting model sees more letters of context, against the uniform baseline and Shannon's estimate for people.

The argument between rules and statistics has a well-known sequence of moves:

- **Chomsky (1957).** In *Syntactic Structures*, the sentences "Colorless green ideas sleep furiously" and "Furiously sleep ideas green colorless" had presumably never occurred in any text. So, Chomsky argued, any statistical model of grammaticalness would treat them as equally remote from English, although only the first is grammatical.
- **Pereira (2000).** Fernando Pereira answered with a calculation: a simple bigram model that groups words into classes, trained on newspaper text, gives the grammatical sentence about 200,000 times the probability of its reversal. A model that shares statistical strength across similar words can assign sensible probabilities to sentences it has never seen, which is the central idea of Weeks 2 and 3.
- **Norvig (2011).** Peter Norvig's essay *On Chomsky and the Two Cultures of Statistical Learning* answers Chomsky's later criticism that statistical models predict without explaining, and is a readable account of the whole debate.

The company a word keeps

Pereira's model worked because it treated similar words alike. Which words are similar? Zellig Harris's answer, in *Distributional Structure* (1954), was: words that occur in similar environments. The *distribution* of a word is the set of environments in which it occurs, and differences of meaning correspond,

he wrote, roughly "to the amount of difference in their environments." Take six sentences: "the oculist examined the eye", "the eye-doctor examined the eye", "the oculist prescribed glasses", "the eye-doctor prescribed glasses", "the lawyer examined the contract", and "the lawyer drafted the contract". Without knowing what any word means, one can see that *oculist* and *eye-doctor* occur in exactly the same environments, while *lawyer* shares only some of them (Figure 9).

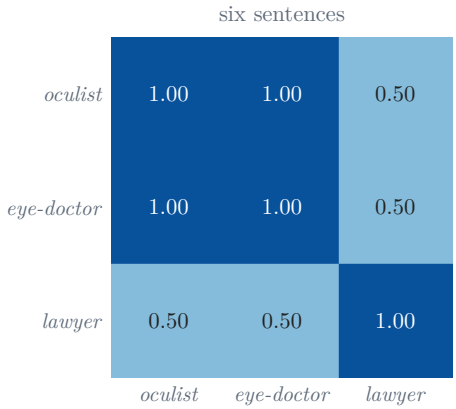


Figure 9. Similarity from environments in the six sentences. Each word is represented by counts of the words that follow it, and the cells show cosine similarities: "oculist" and "eye-doctor" have the same environments, and "lawyer" shares half of them.

J. R. Firth's version became a slogan: "You shall know a word by the company it keeps" (Firth, 1957). The *distributional hypothesis* that grew out of this work underlies word vectors, which represent a word by statistics of the contexts it occurs in (Figure 10; Week 2). Distributional evidence is informative but not the same as meaning: "increase" and "decrease" occur in very similar contexts and mean opposite things.

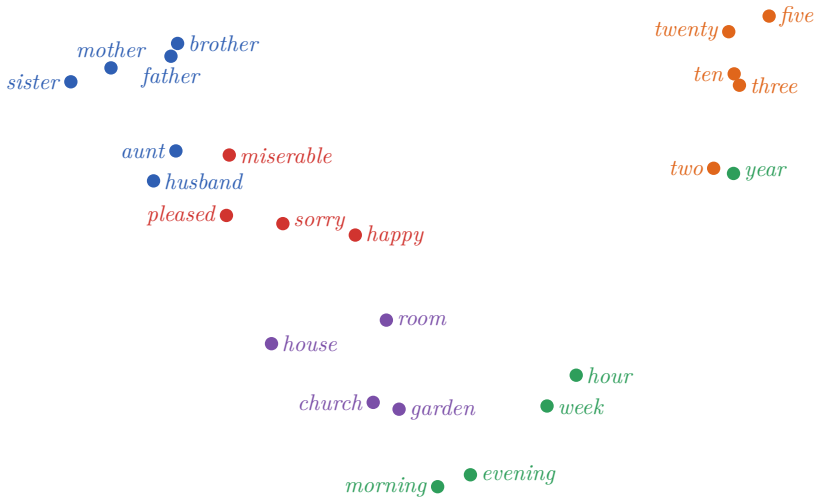


Figure 10. Word vectors from counts in two Austen novels: co-occurrence counts in a small window, reweighted and reduced to two dimensions. Words for relatives, feelings, numbers, places, and times form separate groups.

Neural language models turned these statistics into learned representations. The neural probabilistic language model of Yoshua Bengio and colleagues (2003) learned a vector for each word jointly with a predictor of the next word, so that words used in similar contexts receive similar vectors and share statistical strength. One vector per word is still too coarse: *bank* keeps different company in "the bank of the river" and "the bank raised its rates". Modern models therefore compute a different vector for each occurrence of a word, from the words around it. **Attention**, the central operation of the Transformer (Weeks 3 and 4), does this directly: the new representation of each position is a weighted combination of the positions it reads from, with weights computed from the words themselves. A word is known, quite literally, by the company it keeps in the sentence at hand.

A language model trained to predict the next token is, in this sense, a learned summary of the company every word keeps in its training data. Formal structure, frequency, and context illuminate different aspects of language. Week 2 builds models from the last two: counts, word vectors, and the first neural language models.

7. Meaning: grounding and relations

A language model is trained on form alone: strings of tokens. §6 showed how much structure and statistics the strings contain. This section asks what else meaning involves, and whether it can be learned from form. There are two views. On the first, meaning requires grounding, a connection between words and the world that text does not supply. On the second, much of meaning is relational, and relations can be recovered from text, up to a correspondence with a language that is grounded. The section ends with a consequence of the predictive objective that holds whichever view is right: probable text need not be true.

What words are about

Aristotle's *On Interpretation* separates a name, which signifies by convention, from a statement, which can be true or false only once expressions are combined. Gottlob Frege distinguished an expression's reference (what it picks out) from its sense (the way it presents what it picks out) in *On Sense and Reference* (1892). "The morning star" and "the evening star" both refer to Venus while conveying different information.

The meaning of a whole depends on the meanings of its parts and how they are combined. "Every student read a paper" has two readings, $\forall x \, \exists y \, R(x, y)$ and $\exists y \, \forall x \, R(x, y)$, which differ in truth conditions while using the same words. A representation that records only which words occur cannot distinguish them, which is one reason language models process ordered sequences.

Ludwig Wittgenstein's *Philosophical Investigations* (1953) examines language as a family of activities, such as giving orders, reporting, and asking, and suggests that for a large class of cases the meaning of a word is its use in the language.

Paul Grice explained in *Logic and Conversation* (1975) how speakers communicate more than they literally say. Asked whether all four experiments succeeded, someone who answers "Some succeeded" suggests that not all did, although the answer is logically consistent with all four succeeding; the suggestion can be cancelled by adding "in fact, all four did." An assistant that answers questions must handle this layer of meaning, which is invisible to a purely logical analysis of sentences.

Grice's example suggests a picture of communication. A speaker has something to convey, an intent, and chooses an utterance that, together with the context the two parties share, should lead the listener to it. The listener reconstructs an intent from the utterance and from their own view of the context (Figure 11):

$$\text{intent} \xrightarrow[\text{speaker's context}]{\text{expression}} \text{utterance} \xrightarrow[\text{listener's context}]{\text{interpretation}} \text{inferred intent.}$$

The inferred intent need not equal the original, since the two sides may not share the relevant context. "It's cold in here" can report the temperature, explain why someone is wearing a coat, or ask someone to close the window. Which it does depends on the physical situation, on what was said before, and on what the listener believes about the speaker. The words fix much less than the meaning.

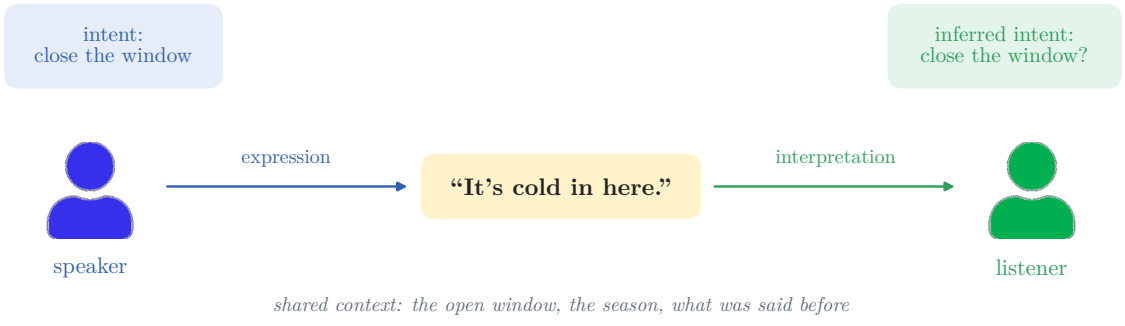


Figure 11. Communication as expression and interpretation. The same utterance can carry different intents; the listener recovers one from the words together with the shared context.

Each of these relates a string to something outside it: to objects and situations in the world, to what a speaker means and is doing, and to a context that speaker and listener share. A language model is trained on text alone. Its objective rewards predicting what a writer wrote next, not recovering what the writer referred to or meant, and no label for the referent or the intent is ever supplied. The text carries evidence about all of these, often a great deal of it. Whether that evidence is enough is the question of the rest of this section.

The grounding problem

Emily Bender and Alexander Koller argue that meaning cannot be learned from form alone (Bender and Koller, 2020). In their thought experiment, two people stranded on separate islands communicate through an underwater cable. A hyperintelligent octopus taps the cable, learns the statistical patterns of their messages, and eventually impersonates one of them convincingly. Then one of the islanders is chased by an angry bear, grabs a couple of sticks, and asks the other for advice on defending herself. The octopus cannot help, because nothing in the patterns of the messages connects words to the objects and situations they are about. This is a version of the older symbol grounding problem: how can the symbols of a formal system mean anything to the system itself, rather than only to the people who interpret it?

Meaning up to isomorphism

The opposing view holds that much of meaning is relational. A dictionary makes the point concrete. Every entry defines a word by a sequence of other words from the same book, so the dictionary as a whole is a large network of words that refer to one another. A newcomer who knows no English cannot learn what the words mean from it alone: every definition leads only to more words, which is the grounding problem again. But suppose the newcomer also has a dictionary of their own language. If the two networks of cross-references have nearly the same shape, the newcomer can match them word for word from their structure alone, and then understand the English words through their own (Figure 12). The relations do not ground the words, but they fix them up to a correspondence with a language that is grounded.

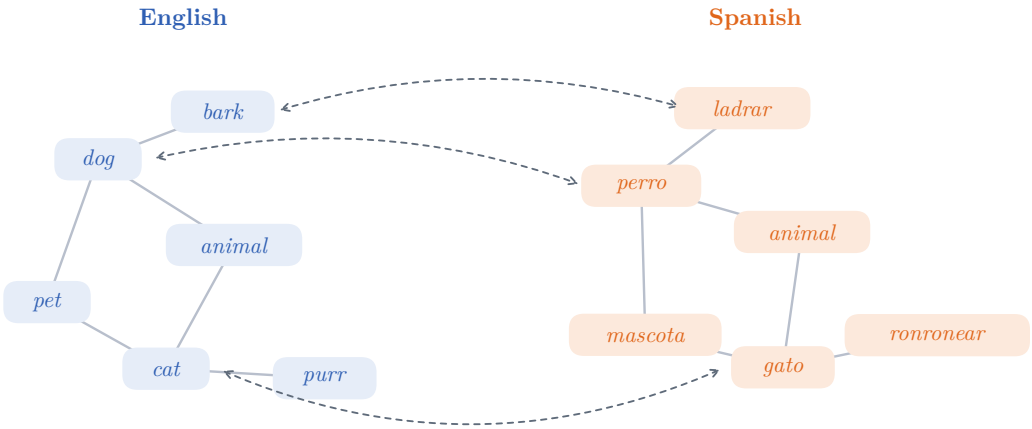


Figure 12. Two dictionaries as networks of cross-references (a few English words and their Spanish counterparts). If the networks have the same shape, the structure alone determines which word corresponds to which.

Something like this happens with word vectors. The vectors of different languages, each learned from text in one language only, have similar geometry. Mikolov, Le, and Sutskever (2013) found that a linear map learned from a small seed dictionary translates many other words, and Conneau and colleagues (2018) learned such a map with no dictionary at all. The match is imperfect, and it degrades for languages and text collections that differ more (Søgaard, Ruder, and Vulić, 2018). Representations learned by different neural models can likewise be translated into one another without any paired examples (Jha and colleagues, 2025).

Steven Piantadosi and Felix Hill argue in this spirit that the meaning of a concept consists in large part of its relations to other concepts, its conceptual role, and that a model trained on text can learn such relations without reference to the world (Piantadosi and Hill, 2022). The question is not settled, and this course does not settle it. It does suggest a more precise formulation. A distribution over text is a very

rich object: its conditional structure encodes grammar, facts stated in text, and the patterns of reasoning humans write down. The question is what information about the world, and about communicative intent, is recoverable from that distribution, and what additional information comes from the later stages of training, in which people demonstrate, compare, and check.

Probable is not true

Shannon's theory of communication, the source of the statistical view of §6, deliberately set meaning aside: messages frequently have meaning, but "these semantic aspects of communication are irrelevant to the engineering problem." Entropy measures uncertainty about which message is sent, not what it says. A language model inherits this stance: its training objective is predictive, and it is rewarded for assigning high probability to observed text whether or not that text is true.

Even perfect statistical modeling then has consequences one might not want. Adam Tauman Kalai and Santosh Vempala consider arbitrary facts, such as which paper a person wrote in a given year, whose truth cannot be inferred from patterns in other facts. They prove that a language model that is calibrated, in the sense that its probabilities match frequencies, must generate false statements of this kind at a rate roughly equal to the fraction of such facts that appear exactly once in its training data, a Good–Turing estimate of the unseen mass (Kalai and Vempala, 2024). The result has a precise scope. It is a lower bound for calibrated models on facts of this arbitrary kind, and it gives no statistical reason to hallucinate facts that appear many times in the data or that follow systematic patterns, such as arithmetic. For the facts it does cover, hallucination is a consequence of calibrated prediction rather than an engineering defect, so reducing it means giving up some calibration, which post-training (Week 7) can do, or consulting an external source such as retrieved documents.

8. Hands-on: an automatic-differentiation engine

Every neural model in this course is trained by gradient methods, and every gradient is computed by reverse-mode automatic differentiation. The lecture's hands-on segment works through an engine that does this, written from scratch in a few hundred lines of plain Python and NumPy: `tinylm/autograd.py`. Part A of the Week 1 notebook follows this section step by step; Part B measures the cost of gradients and trains two classifiers with the engine.

Tensors and computation graphs

A **tensor** here is an array together with what is needed to differentiate through it: its data, a gradient of the same shape that the backward pass fills in, the tensors it was computed from (its parents), and a rule for passing gradients back to them. Every operation creates a new tensor and records these. Evaluating an expression such as $L = \text{mean}\left((XW + b)^2\right)$ therefore builds a directed acyclic graph whose leaves are the inputs and parameters and whose root is the scalar loss (Figure 13).

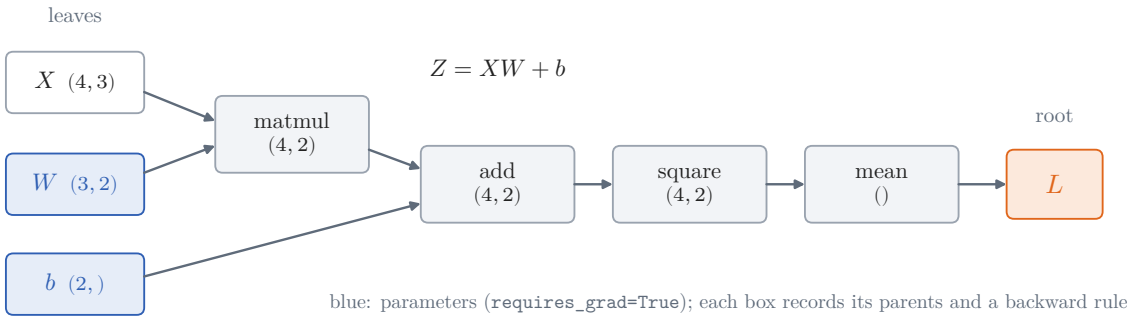


Figure 13. The computation graph of $L = \text{mean}\left((XW + b)^2\right)$ for 4 examples with 3 features and 2 outputs. Each box is a tensor, labeled with the operation that produced it and its shape; the leaves W and b are the parameters.

The backward pass

Backpropagation computes the gradient of a scalar loss with respect to every parameter of a composed computation by applying the chain rule backwards through the graph. Each operation contributes one

local rule. If an operation maps an input u to an output z, and $\bar{z} = \partial L/\partial z$ is already known, the rule returns $\bar{u} = (\partial z/\partial u)^\top \bar{z}$, a **vector–Jacobian product**. The backward pass visits the graph in reverse topological order, so that each node is processed only after every node that uses it, and applies these rules from the root to the leaves. It runs the forward graph in reverse, so the architecture chosen for the forward computation also decides how the learning signal reaches each parameter.

Two details carry most of the subtlety. First, when a value is used more than once, its gradient is the **sum** of the contributions from its uses; this is the multivariable chain rule. In $f(x, y) = \log(1 + e^{xy}) + x^2$ the variable x is used twice, and

$$\frac{\partial f}{\partial x} = y\,\sigma(xy) + 2x, \qquad \sigma(t) = \frac{1}{1 + e^{-t}},$$

one term for each use. At $(x, y) = (1.5, -0.5)$ the two contributions are -0.160 and 3.000, and the backward pass adds them (Figure 14).

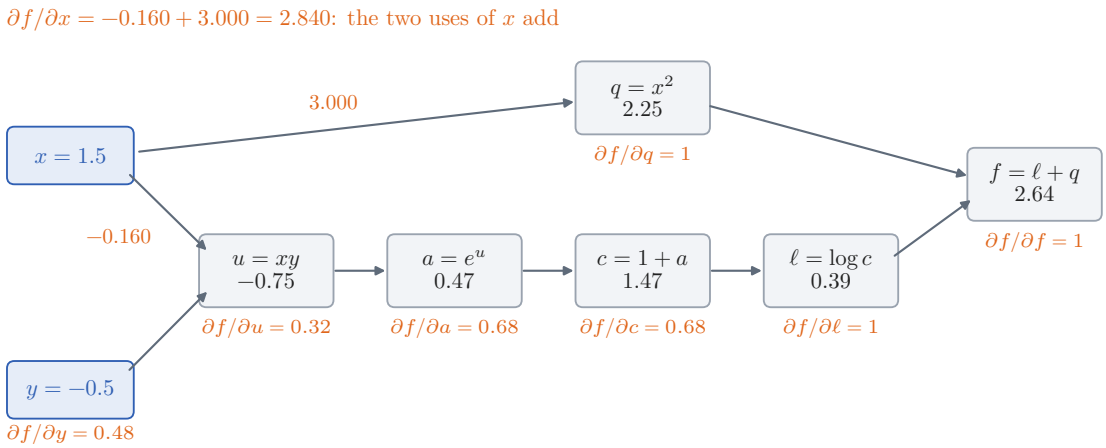


Figure 14. The backward pass for $f(x, y) = \log(1 + e^{xy}) + x^2$ at $x = 1.5$, $y = -0.5$. Forward values are in blue and gradients in orange. The two uses of x send back -0.160 and 3.000, which add to 2.840.

Second, **broadcasting is reuse**. In $Z = XW + b$, with X of shape $n \times d$ and W of shape $d \times k$, the bias $b \in \mathbb{R}^k$ is added to each of the n rows, so its gradient is the sum of the rows of $G = \partial L/\partial Z$. The matrix product has the rules $\partial L/\partial W = X^\top G$ and $\partial L/\partial X = GW^\top$. In every case the gradient has the shape of the tensor it belongs to (Figure 15), which is a useful check when deriving rules by hand.

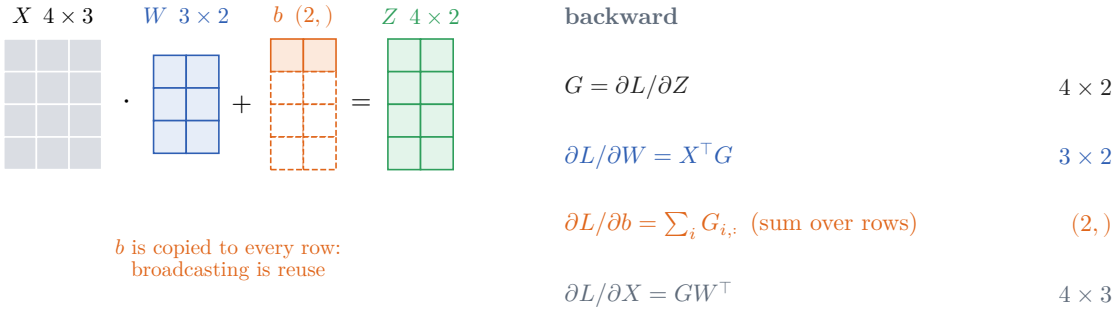


Figure 15. Gradient shapes for $Z = XW + b$. The bias b is copied to every row, so its gradient sums the rows of G; every gradient has the shape of its tensor.

Checking gradients

A mistake in a backward rule rarely causes a crash. It silently produces a wrong gradient, and training quietly does worse. Every rule in the engine is therefore checked against the central difference

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h}.$$

Its truncation error is of order h^2, while rounding in the two evaluations of f contributes about ϵ_{mach}/h, so the total is smallest near $h \approx \epsilon_{\text{mach}}^{1/3} \approx 10^{-5}$ in double precision (Figure 16). The engine's tests also compare its gradients with those of PyTorch.

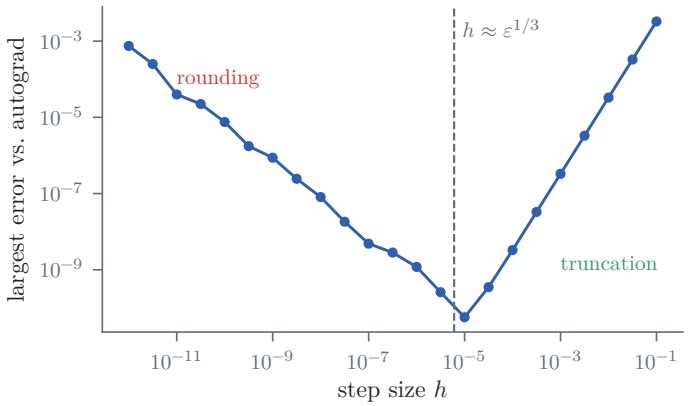


Figure 16. Error of the central difference against the step size h, for a small network. Rounding error dominates for small h and truncation error for large h; the best h is near 10^{-5}.

Three ways to compute a derivative

Why build an engine at all, when a derivative can also be worked out as a formula or estimated from function values? Figure 17, from the survey of Baydin, Pearlmutter, Radul, and Siskind (2018), compares the three ways on one small program: the logistic map $l_1 = x$, $l_{n+1} = 4l_n(1 - l_n)$, run for three steps to give $f(x) = l_4$.

- **Symbolic differentiation** works on formulas, not programs. It is exact, but the program must first be expanded into a single formula, and a value that the program uses twice appears twice in the formula. The logistic map uses l_n twice at every step, so the formula doubles with each step, and its derivative, by the product rule, grows faster still. After seven steps the program performs 21 arithmetic operations, while its derivative written out as a formula has about 11,000 nodes, and each further step multiplies that by more than 2.5. A program whose loops and branches depend on the data has no single formula at all.
- **Numerical differentiation** needs nothing but the ability to evaluate f, which is why it is the standard check of the previous subsection. It has two costs. It is approximate: the step h trades truncation error against rounding error (Figure 16), so even the central difference keeps only about two thirds of the digits of double precision, four or five in single precision, and one or two in the 16-bit formats used to train large models. And it measures one direction at a time: a gradient with respect to n inputs needs $n + 1$ evaluations of f with forward differences and $2n$ with central ones.
- **Automatic differentiation** applies the exact derivative of each elementary operation to the values the program actually computes, and combines them by the chain rule. Nothing is expanded: the derivative computation is a constant factor larger than the program, and it follows whichever loops and branches the program took. Its result is exact up to rounding errors of the same size as those made in evaluating f.

Against symbolic differentiation, then, automatic differentiation wins on size, because it keeps the program's shared intermediate values instead of copying them into a formula. Against numerical differentiation it wins twice. The gain in accuracy is the smaller win, since gradient descent tolerates some noise in its gradients. The decisive win is cost, and it depends on the direction in which the chain rule is applied.

Why reverse mode

The derivative program in Figure 17 runs in **forward mode**: it carries the derivative with respect to one input forward, alongside the values. One such pass gives the derivative in one input direction, so a gradient with respect to n inputs still takes n passes, the same scaling as finite differences. For the logistic map, with a single input, this is no drawback; for a loss that depends on millions of parameters, it is the whole difference.

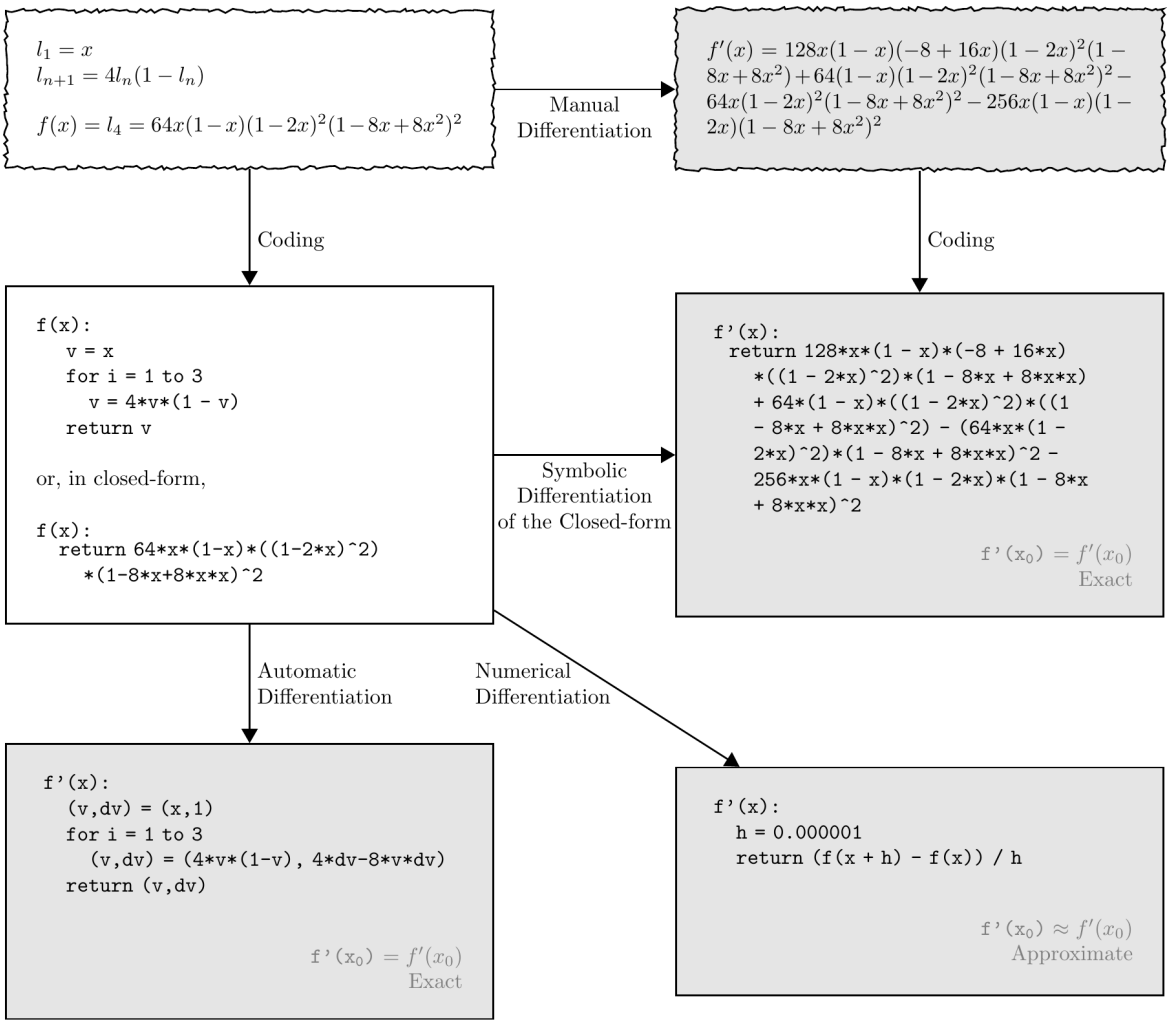


Figure 17. Three routes from a program to its derivative, for $f(x) = l_4$ (after Baydin et al., 2018, their Fig. 2). Differentiating the closed-form formula, by hand or symbolically, gives an exact but long expression. Numerical differentiation evaluates the program twice and gives an approximation. Automatic differentiation transforms the program itself, here by carrying each value v together with its derivative dv, and is exact up to rounding.

Reverse mode, the backward pass of this section, applies the chain rule in the other direction. It starts from the single output and carries its sensitivity back to every input at once. The gradient with respect to all n inputs then costs only a small constant factor more arithmetic than evaluating the function once, whatever n is (Baur and Strassen, 1983). The price is memory: the forward pass must keep its intermediate values until the backward pass has used them. In general, for a function with n inputs and m outputs, forward mode needs about n passes and reverse mode about m; a loss has $m = 1$.

For a model with 100 million parameters, one finite-difference gradient needs 200 million evaluations of the loss, while a forward and a backward pass together cost about as much as two or three evaluations. For the width-1024 network of Part B of the notebook, that is about 15 minutes against about ten milliseconds. This gap in cost, far more than the digits of accuracy, is why networks with billions of parameters can be trained at all. Foundations 3 develops both modes in detail.

Gradients and updates

Backpropagation computes derivatives; the **optimizer** decides how to use them. Gradient descent (§5) moves each parameter a small step against its gradient; the variants of Week 5 adapt the step using the history of gradients. Because gradients accumulate across backward passes, a training loop clears them before each step, then runs the forward pass, the backward pass, and the update. The same loop, with a cross-entropy loss and a small network, trains the classifiers in Part B of the notebook.

From here to language models

A language model applies these ingredients to the distribution over strings of §2. Each conditional $q_\theta(x_t \mid x_{<t})$ is a classifier over the vocabulary, computed by a network from the prefix; training minimizes the log loss of observed next tokens, which is self-supervised; and generation samples one token at a time, feeding each back as input. The lifecycle of a modern model has four stages, each with its own source of learning signal:

Stage	Signal	What changes	Weeks
Pretraining	Next tokens of a large text corpus	All parameters, from a random start	2–5
Supervised fine-tuning	Demonstrations of desired behavior	Parameters, from the pretrained model	7
Preference and RL post-training	Comparisons and rewards	Parameters, relative to a reference model	6–7
Inference	None: prompts, retrieved text, and tool outputs condition a fixed model	Nothing in the parameters	5, 8

The last row is a distinction worth keeping in mind from the start. A model can appear to learn during a conversation, because a fact placed in its context changes its predictions. That is conditioning, not learning in the sense of §3: nothing persists once the context is gone.

Next week

We now have the vocabulary to ask precise questions about the model we will build. Week 2 starts with the simplest language models and three questions: how to assign probabilities to strings, how to measure whether a predictor is any good, and why counting alone runs out. We answer them with counts, then with the first neural language models.

9. Background reading

The basics of §§3, 5, and 8 are developed in full elsewhere. The table lists where: the chapters of the Background Notes on Machine Learning and AI, and standard textbooks.

Topic	Background Notes	Textbook
Terminology; the objects being learned	Foundations 1	Russell and Norvig, Chapters 1–2
Linear algebra and calculus	Foundations 2 and 3	Goodfellow, Bengio, and Courville, Chapters 2 and 4
Probability, likelihood, and Bayes' rule	Foundations 4; ML 4 and 5	Goodfellow, Bengio, and Courville, Chapter 3
Risk, generalization, and PAC learning	Foundations 5; ML 7	Shalev-Shwartz and Ben-David, Chapters 2–6
Information theory	Foundations 5	Cover and Thomas, Chapter 2
Evaluation and data leakage	ML 6	
The perceptron	ML 2	Shalev-Shwartz and Ben-David, Chapter 9
Neural networks and backpropagation	DL 1; Foundations 3	Goodfellow, Bengio, and Courville, Chapter 6
Optimization and regularization	DL 3 and 5	Goodfellow, Bengio, and Courville, Chapters 7–8
Search, logic, and probabilistic reasoning	AI 1–9	Russell and Norvig, Parts II–IV
Numerical computing with arrays	Foundations 6	

For reading in the order of this lecture: Turing's 1950 paper is short and still rewarding; Valiant's 1984 paper introduces the PAC model in eight pages; and Chapters 2 and 3 of Shalev-Shwartz and Ben-David develop the learning theory.

References and further reading

History

- **Alan M. Turing (1936), On Computable Numbers, with an Application to the Entscheidungsproblem.** Paper.
- **Arthur L. Samuel (1959), Some Studies in Machine Learning Using the Game of Checkers.** Paper.
- **Murray Campbell, A. Joseph Hoane Jr., and Feng-hsiung Hsu (2002), Deep Blue.** Paper.
- **Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton (2012), ImageNet Classification with Deep Convolutional Neural Networks.** Paper.
- **Ian Goodfellow et al. (2014), Generative Adversarial Networks.** Paper.
- **David Silver et al. (2016), Mastering the Game of Go with Deep Neural Networks and Tree Search.** Paper.
- **Ashish Vaswani et al. (2017), Attention Is All You Need.** Paper.
- **Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever (2018), Improving Language Understanding by Generative Pre-Training.** Report. **Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2018), BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.** Paper.
- **Tom B. Brown et al. (2020), Language Models are Few-Shot Learners.** Paper.
- **OpenAI (2022), Introducing ChatGPT.** Announcement. **OpenAI (2023), GPT-4 Technical Report.** Report. **OpenAI (2024), Hello GPT-4o.** Announcement. **OpenAI (2024), Introducing OpenAI o1-preview.** Announcement.
- **DeepSeek-AI (2025), DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.** Paper.
- **Google DeepMind (2025), Advanced Version of Gemini with Deep Think Officially Achieves Gold-Medal Standard at the International Mathematical Olympiad.** Report.
- **Terence Tao and contributors, AI Contributions to Erdős Problems.** Tracker. **Resolution of Erdős Problem #728: A Writeup of Aristotle's Lean Proof (2026).** Paper. **Quanta Magazine (2026), Why the Legendary Erdős Problems Are Falling to AI.** Article.
- **OpenAI (2026), An OpenAI Model Has Disproved a Central Conjecture in Discrete Geometry.** Announcement. **Remarks on the Disproof of the Unit Distance Conjecture (2026).** Paper.
- **OpenAI (2026), On the Navier–Stokes Millennium Prize Problem.** Announcement. **Clay Mathematics Institute (2026), Navier–Stokes Announcement.** Statement. **Quanta Magazine (2026), AI Has Solved One of Math's $1 Million Millennium Prize Problems.** Article.

Intelligence

- **René Descartes (1637), Discourse on the Method.** Text. Part V on machines and language.
- **Alan M. Turing (1950), Computing Machinery and Intelligence.** Paper. The imitation game, objections to machine intelligence, and learning machines.
- **John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon (1955), A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.** Proposal.
- **Allen Newell and Herbert A. Simon (1976), Computer Science as Empirical Inquiry: Symbols and Search.** Paper. The physical symbol system hypothesis.
- **Warren McCulloch and Walter Pitts (1943), A Logical Calculus of the Ideas Immanent in Nervous Activity.** Paper.

- **Frank Rosenblatt (1958), The Perceptron.** Paper. A film clip of the Mark I Perceptron.
- **Joseph Weizenbaum (1966), ELIZA: A Computer Program for the Study of Natural Language Communication between Man and Machine.** Paper.
- **Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th edition (2020).** Book site. Definitions of AI and the rational-agent view.
- **Shane Legg and Marcus Hutter (2007), Universal Intelligence: A Definition of Machine Intelligence.** Paper.
- **François Chollet (2019), On the Measure of Intelligence.** Paper.
- **John R. Searle (1980), Minds, Brains, and Programs.** Paper. The Chinese room.
- **Richard Sutton (2019), The Bitter Lesson.** Essay. **Richard Sutton and Dwarkesh Patel (2025), Father of RL Thinks LLMs Are a Dead End.** Podcast; video.

Learning

- **David Hume (1748), An Enquiry Concerning Human Understanding.** Text. Sections IV and V on induction.
- **David H. Wolpert (1996), The Lack of A Priori Distinctions Between Learning Algorithms.** Paper. No free lunch.
- **Albert B. J. Novikoff (1962), On Convergence Proofs on Perceptrons.** Proceedings of the Symposium on the Mathematical Theory of Automata, 615–622. The mistake bound for the perceptron.
- **Welch Labs, Why Deep Learning Works Unreasonably Well (How Models Learn, Part 3).** Video.
- **Leslie G. Valiant (1984), A Theory of the Learnable.** Paper. The PAC model.
- **Shai Shalev-Shwartz and Shai Ben-David (2014), Understanding Machine Learning: From Theory to Algorithms.** Book. Chapters 2–6 for PAC learning, uniform convergence, and VC dimension.
- **Michael Kearns and Leslie G. Valiant (1994), Cryptographic Limitations on Learning Boolean Formulae and Finite Automata.** Paper.
- **Dana Angluin (1987), Learning Regular Sets from Queries and Counterexamples.** Paper. The L* algorithm.
- **Ray J. Solomonoff (1964), A Formal Theory of Inductive Inference, Part I.** Paper.
- **Jorma Rissanen (1978), Modeling by Shortest Data Description.** Paper. Minimum description length.
- **Grant Sanderson (2026), Compression is Intelligence, Parts 1 and 2: Reinventing Entropy; But What Is Cross-Entropy?** Part 1; Part 2.
- **Nathan Barry (2026), Can gzip Be a Language Model?** Essay.

Language and meaning

- **Pāṇini, Aṣṭādhyāyī.** Śrīśa Chandra Vasu's English translation, volume I (1891).
- **Aristotle, On Interpretation.** E. M. Edghill's translation, chapters 1–4.
- **Gottlob Frege (1892), On Sense and Reference.** English translation.
- **Ludwig Wittgenstein (1953), Philosophical Investigations.** Published edition, especially sections 2, 23, and 43.
- **H. P. Grice (1975), Logic and Conversation.** Chapter.
- **Zellig S. Harris (1954), Distributional Structure.** *Word* 10(2–3), 146–162. DOI; PDF. Section 2.3 on meaning (the *oculist* example) and Section 4 on relations between neighboring sentences.
- **J. R. Firth (1957), A Synopsis of Linguistic Theory, 1930–1955.** Essay.
- **Noam Chomsky (1956), Three Models for the Description of Language.** Paper. Also *Syntactic Structures* (1957), Chapter 2, for "colorless green ideas".
- **Noam Chomsky (1959), Review of B. F. Skinner's Verbal Behavior.** Paper.
- **E. Mark Gold (1967), Language Identification in the Limit.** Paper.

- **James J. Horning (1969), A Study of Grammatical Inference.** PhD thesis, Stanford University.
- **Dana Angluin (1980), Inductive Inference of Formal Languages from Positive Data.** Paper.
- **Jon Kleinberg and Sendhil Mullainathan (2024), Language Generation in the Limit.** Paper.
- **Alkis Kalavasis, Anay Mehrotra, and Grigoris Velegkas (2025), On the Limits of Language Generation: Trade-Offs Between Hallucination and Mode Collapse.** Paper.
- **Jon Kleinberg and Fan Wei (2025), Density Measures for Language Generation.** Paper.
- **Claude E. Shannon (1948), A Mathematical Theory of Communication.** Paper. Section 3 of Part I for the approximations to English.
- **Claude E. Shannon (1951), Prediction and Entropy of Printed English.** Paper.
- **Fernando Pereira (2000), Formal Grammar and Information Theory: Together Again?** Paper.
- **Peter Norvig (2011), On Chomsky and the Two Cultures of Statistical Learning.** Essay.
- **Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin (2003), A Neural Probabilistic Language Model.** Paper.
- **Michael C. Frank and Noah D. Goodman (2012), Predicting Pragmatic Reasoning in Language Games.** Paper.
- **Noga Zaslavsky, Charles Kemp, Terry Regier, and Naftali Tishby (2018), Efficient Compression in Color Naming and Its Evolution.** Paper.
- **Emily M. Bender and Alexander Koller (2020), Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data.** Paper.
- **Stevan Harnad (1990), The Symbol Grounding Problem.** Paper.
- **Tomas Mikolov, Quoc V. Le, and Ilya Sutskever (2013), Exploiting Similarities among Languages for Machine Translation.** Paper.
- **Alexis Conneau, Guillaume Lample, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou (2018), Word Translation Without Parallel Data.** Paper.
- **Anders Søgaard, Sebastian Ruder, and Ivan Vulić (2018), On the Limitations of Unsupervised Bilingual Dictionary Induction.** Paper.
- **Rishi Jha, Collin Zhang, Vitaly Shmatikov, and John X. Morris (2025), Harnessing the Universal Geometry of Embeddings.** Paper.
- **Steven T. Piantadosi and Felix Hill (2022), Meaning without Reference in Large Language Models.** Paper.
- **Adam Tauman Kalai and Santosh S. Vempala (2024), Calibrated Language Models Must Hallucinate.** Paper.

Basics

- **Andrew Y. Ng and Michael I. Jordan (2001), On Discriminative vs. Generative Classifiers.** Paper.
- **Walter Baur and Volker Strassen (1983), The Complexity of Partial Derivatives.** Paper.
- **Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind (2018), Automatic Differentiation in Machine Learning: a Survey.** *Journal of Machine Learning Research* 18(153), 1–43. Paper.
- **David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams (1986), Learning Representations by Back-Propagating Errors.** Paper.
- **George Cybenko (1989), Approximation by Superpositions of a Sigmoidal Function.** Paper. **Kurt Hornik (1991), Approximation Capabilities of Multilayer Feedforward Networks.** Paper.
- **Yann LeCun, Yoshua Bengio, and Geoffrey Hinton (2015), Deep Learning.** Review.
- **Ian Goodfellow, Yoshua Bengio, and Aaron Courville (2016), Deep Learning.** Book.

- **Thomas M. Cover and Joy A. Thomas, Elements of Information Theory, 2nd edition (2006).** Book. Chapters 2 and 5 on entropy and coding.

Image credits

Photographs and the Transformer diagram come from Wikimedia Commons, Figures 3h and 3i from OpenAI, Figure 5 from the Dwarkesh Podcast, and Figure 17 from Baydin et al. (2018). All other figures were made for these notes; the language statistics in Figures 8 and 10 use two public-domain novels by Jane Austen from NLTK's selection of Project Gutenberg texts.

- Figure 1a: Campfire flames at night.jpg, Marc-Lautenbacher, CC BY-SA 4.0.
- Figure 1b: Ljubljana Marshes Wheel with axle (oldest wooden wheel yet discovered).jpg, Petar Milošević, CC BY-SA 4.0.
- Figure 1c: Boulton & Watt steam engine, Sydney Powerhouse Museum, 2014 (15243331353).jpg, David Maciulaitis from Auckland, New Zealand, CC BY 2.0.
- Figure 1d: Cuneiform tablet- student exercise tablet MET DP360672.jpg, The Metropolitan Museum of Art, CC0.
- Figure 1e: Bakhshali manuscript zero detail.jpg, Bodleian Library manuscript, Public domain.
- Figure 1f: Gutenberg press.png, International Printing Museum, CC0.
- Figure 1g: Morse's telegraph station, 1931 - Morse's telegraph devices introduced in Serbia in 1855.jpg, Milica Buha, CC BY-SA 4.0.
- Figure 2b: Eniac.jpg, U.S. Army, Public domain.
- Figure 2c: A replica of the first working transistor 02.jpg, Mister rf, CC BY-SA 4.0.
- Figure 2f: Mark I Perceptron, Figure 2 of operator's manual.png, John C. Hay et al., Mark I Perceptron Operators' Manual (1960), Public domain.
- Figure 2i: Deep Blue.jpg, James the photographer, CC BY 2.0.
- Figure 2l: Go (13×13) – 2021 – 6741.jpg, Dietmar Rabich, CC BY-SA 4.0.
- Figure 3a: Transformer, full architecture.png, dvgodoy, CC BY 4.0.
- Figure 3e: Verschiedenfarbige Schwertlilie (Iris versicolor)-20200603-RM-100257.jpg, Ermell, CC BY-SA 4.0.
- Figure 3h: the rescaled square-grid construction, from OpenAI's announcement of the unit distance result (May 2026).
- Figure 3i: a snapshot of local incompressible motion, from OpenAI's announcement of the Navier–Stokes result (September 2026).
- Figure 5: thumbnail of the Dwarkesh Podcast episode with Richard Sutton (2025).
- Figure 17: Figure 2 of A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic Differentiation in Machine Learning: a Survey*, JMLR 18(153), 2018, CC BY 4.0; page 5 of the published PDF, cropped above the caption, artwork unchanged.

Appendix A: deeper reading

This appendix collects the formal results that §§4 and 5 describe in words. The lecture does not cover them.

A.1 Universal intelligence

Shane Legg and Marcus Hutter define the universal intelligence of an agent π as its expected performance across all computable environments, weighted by simplicity:

$$\Upsilon(\pi) = \sum_{\mu \in E} 2^{-K(\mu)} \, V_\mu^\pi .$$

Here E is the class of computable reward-generating environments, V_μ^π is the expected total reward the agent obtains in environment μ, and $K(\mu)$ is the Kolmogorov complexity of μ: the length of the

shortest program that implements it on a fixed universal machine (Legg and Hutter, 2007). The definition explains a common intuition, that intelligence is the ability to do well across many environments with a bias toward simple ones. It is also incomputable, since K is incomputable and the sum ranges over all computable environments, so it cannot be used as a benchmark directly.

A.2 The Occam bound

Here is the proof of the Occam bound of §5. Suppose the learner searches a finite hypothesis class $\mathcal{H} \supseteq \mathcal{C}$ and outputs any hypothesis that agrees with every training example. Call a hypothesis bad if its error exceeds ε. A fixed bad hypothesis agrees with one random example with probability at most $1-\varepsilon$, so it agrees with all m independent examples with probability at most

$$(1-\varepsilon)^m \le e^{-\varepsilon m}.$$

There are at most $|\mathcal{H}|$ bad hypotheses, so by the union bound the probability that any of them survives is at most $|\mathcal{H}|e^{-\varepsilon m}$. Requiring this to be at most δ gives the sample size

$$m \;\ge\; \frac{1}{\varepsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right).$$

The number of examples needed grows with the logarithm of the number of hypotheses, that is, with the number of bits needed to name a hypothesis. For example, with $|\mathcal{H}| = 2^{20}$, $\varepsilon = 0.05$, and $\delta = 0.01$, the bound requires $m \ge 20(20\ln 2 + \ln 100)$, about 370 examples. If $\mathcal{H}$ contains all Boolean functions on n input bits, then $\ln|\mathcal{H}| = 2^n \ln 2$, and the bound becomes exponential in n: the no-free-lunch phenomenon reappears as a sample-complexity statement. Infinite classes are handled by replacing $\ln|\mathcal{H}|$ with a combinatorial measure of complexity such as the VC dimension. The general theory, including the case where no hypothesis fits the data perfectly, is developed in Shalev-Shwartz and Ben-David and in ML 7 of the Background Notes.

A.3 Hardness and queries

The Occam bound says nothing about time. Finding a hypothesis consistent with the data can be computationally hard even when few examples suffice, and for some classes no efficient learner exists at all under standard cryptographic assumptions. Michael Kearns and Leslie Valiant (1994) showed that deterministic finite automata cannot be efficiently PAC learned, even if the learner may output any polynomially evaluable hypothesis, unless certain cryptographic problems (such as inverting RSA) are easy. The intuition is that a finite automaton can compute a pseudorandom-looking function; if one could learn it from random examples, one could predict the output of a cryptographic generator.

The hardness depends on how the learner obtains information. Dana Angluin's L* algorithm learns any regular language exactly, in time polynomial in the number of states of its minimal automaton and the length of the longest counterexample it receives, if the learner may ask two kinds of question: membership queries ("is this string in the language?") and equivalence queries ("is this automaton correct? if not, give me a counterexample") (Angluin, 1987). The contrast between the two results is the one drawn in §5: what a learner can learn efficiently depends on the channel through which it gets information.

These results concern automata, not neural networks trained by gradient descent, and they are worst-case statements. They should be read as warnings about what cannot be guaranteed in general, not as predictions about what happens on natural language. The gap between worst-case hardness and empirical success recurs throughout the course.

A.4 Solomonoff induction

Ray Solomonoff's idealized solution to induction (Solomonoff, 1964) considers all programs for a fixed universal machine and gives each program p prior weight $2^{-|p|}$, so that shorter programs are more probable. The universal prior probability of a finite string x is the total weight of programs whose output begins with x. The next symbol is predicted by conditioning this prior on what has been seen. Solomonoff later proved that this predictor is optimal in a strong sense: for any computable data source μ, its total expected prediction error over an infinite sequence is bounded by a constant proportional to the complexity $K(\mu)$ of the source. Solomonoff induction is incomputable, like the Legg–Hutter measure built on it.

Appendix B: before the deep learning revival

The list of §1 jumps from backpropagation (1986) and Deep Blue (1997) to AlexNet (2012). The field did not stand still in between; it became statistical. Most learning systems of the period combined features designed by hand with a model that was simple to fit, preferably by convex optimization and with a guarantee about new data. Neural networks were out of fashion, though never abandoned. Figure 18 places the developments described below on one time line. The lecture does not cover this appendix.

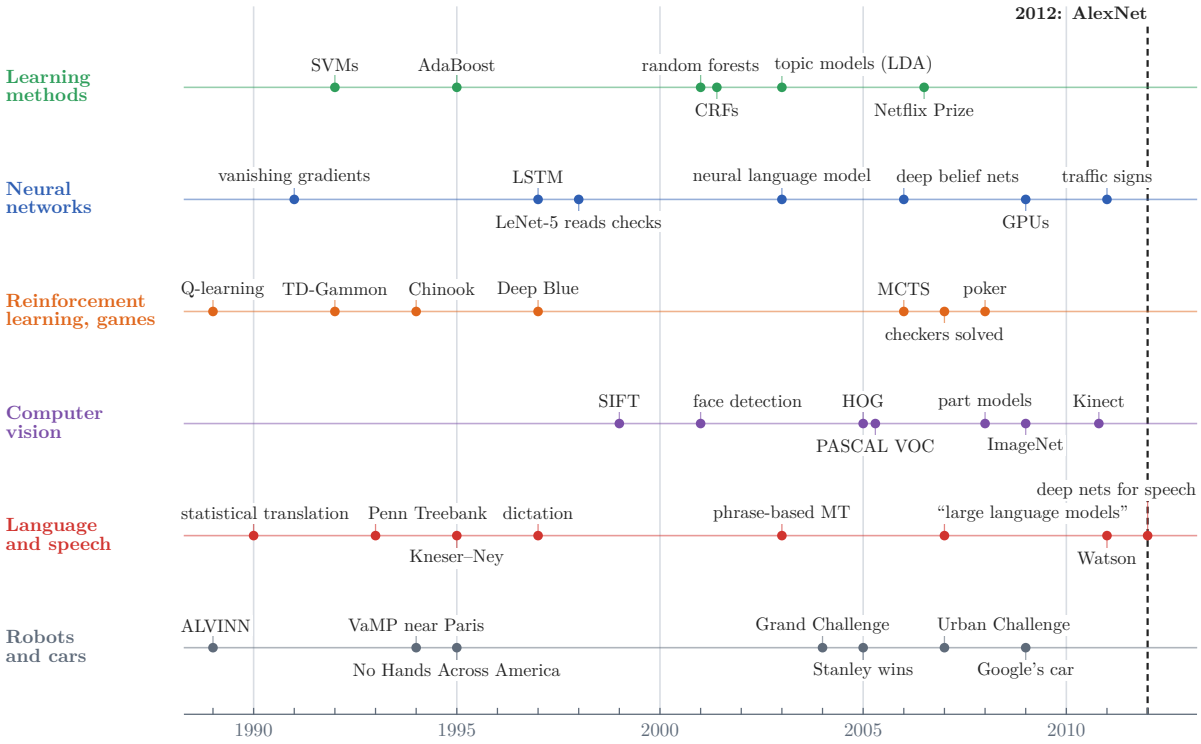


Figure 18. Selected developments from 1989 to 2012, by area; the dashed line marks AlexNet (§1, item 9). Each development is described in the subsections below.

B.1 Why neural networks fell out of favor

After 1986, networks trained by backpropagation worked on real but small problems: they read handwritten digits, steered a van (§B.6), and played backgammon (§B.3). Deeper networks were hard to train. Sepp Hochreiter's 1991 diploma thesis and Bengio, Simard, and Frasconi (1994) showed that gradients passed back through many layers, or many time steps of a recurrent network, tend to vanish or explode, so that the early layers learn slowly or not at all. Computers were slow, labeled data sets were small, and the training problem was not convex, so nothing guaranteed that gradient descent would find a good solution. Methods with convex training and a theory of generalization matched networks on many benchmarks of the day, and most of the field moved to them.

Artificial intelligence as a whole was recovering from the collapse of the expert-systems industry in the late 1980s. Much of its work in the 1990s was recast in terms of probability and rational agents: the view of Russell and Norvig's textbook (1995, §4) and of Judea Pearl's calculus of probabilistic and causal reasoning, which received the 2011 Turing Award. Search and logic advanced too: satisfiability solvers such as Chaff (2001) became fast enough to verify hardware and to serve as general-purpose engines for planning and scheduling.

B.2 Machine learning becomes statistical

The **support vector machine** combined a maximum-margin linear classifier with kernels (Boser, Guyon, and Vapnik, 1992; Cortes and Vapnik, 1995). Training is a convex quadratic program, and Vladimir Vapnik's statistical learning theory (1995) bounds the error on new data in terms of the margin and the capacity of the model class, in the spirit of the PAC bounds of §5. Ensembles of simple models

were the other workhorse: AdaBoost (Freund and Schapire, 1995–1997) trains each new classifier on a reweighting of the data that emphasizes the examples the previous ones got wrong, and random forests (Breiman, 2001) average many randomized decision trees. Data with structure was modeled by probabilistic graphical models: hidden Markov models for speech, conditional random fields (Lafferty, McCallum, and Pereira, 2001) for labeling the words of a sentence, and topic models such as latent Dirichlet allocation (Blei, Ng, and Jordan, 2003) for collections of documents. Sparse estimation (the lasso, 1996) and bandit algorithms with finite-time guarantees (UCB1, 2002) complete the picture of the period: models fitted by convex optimization or by counting, with theorems about their behavior.

These methods were deployed at scale. Statistical spam filters, which combine the evidence of individual words, spread after Paul Graham's essay *A Plan for Spam* (2002). Web search ranked pages by the link analysis of PageRank (1998) and later by rankers learned from data. The Netflix Prize (2006–2009) offered a million dollars for a 10% improvement in the error of Netflix's own predictions of movie ratings. The winners blended many models, and a central ingredient was matrix factorization, which represents every user and every movie by a learned vector and predicts a rating by their inner product: learned embeddings of users and movies.

B.3 Reinforcement learning and games

Reinforcement learning took its modern form in this period: Christopher Watkins's Q-learning (1989, with a convergence proof by Watkins and Peter Dayan in 1992), Ronald Williams's policy-gradient algorithm REINFORCE (1992), the policy gradient theorem of Sutton, McAllester, Singh, and Mansour (1999), and Sutton and Barto's textbook (1998). Its most striking success was Gerald Tesauro's TD-Gammon (1992–1995). A neural network learned to evaluate backgammon positions by temporal-difference learning from games against itself and reached strong master level, close to the best human players. TD-Gammon is a direct ancestor of AlphaGo (§1, item 11) and of the reward-based training of Weeks 6 and 7.

Games were also won by search. Jonathan Schaeffer's checkers program Chinook held the man–machine world title from 1994, and in 2007 his team solved checkers: with perfect play by both sides the game is a draw. Monte Carlo tree search (Coulom, 2006; the UCT algorithm of Kocsis and Szepesvári, 2006) evaluates a move by playing many random games to the end and concentrates the games on promising moves. It lifted computer Go to strong amateur level, and in 2008 the program MoGo beat a professional player who gave it a nine-stone handicap. AlphaGo kept the tree search and added learned networks to choose and evaluate its moves. In poker, a game of hidden information, counterfactual regret minimization (2007) computed approximate equilibrium strategies, and the University of Alberta's Polaris beat professionals at heads-up limit hold'em in 2008. Beyond games, learning from demonstrations (Abbeel and Ng, 2004) let an autonomous helicopter fly aerobatic maneuvers learned from an expert pilot (Abbeel, Coates, and Ng, 2010). Most of this work used tables, or linear functions of features designed by hand; reinforcement learning with deep networks waited for DQN (2013).

B.4 Computer vision

A vision system of the period was a pipeline: features designed by hand, pooled into a vector of fixed length, and passed to a linear classifier, usually a support vector machine. Viola and Jones (2001) detected faces in real time with simple rectangle features, chosen by AdaBoost and arranged in a cascade that discards most of an image after a few tests; detectors of this kind were widely adopted, including in consumer cameras. David Lowe's SIFT (1999, 2004) describes the neighborhood of a keypoint by histograms of gradient directions, so that the same point can be matched across changes of scale, rotation, and lighting; histograms of oriented gradients (Dalal and Triggs, 2005) apply the idea to a dense grid, for detecting pedestrians. For recognition, local descriptors were quantized into "visual words" and counted, just as a bag of words counts the words of a document (Sivic and Zisserman, 2003; Csurka and colleagues, 2004; Lazebnik, Schmid, and Ponce, 2006), and objects were located by deformable part models (Felzenszwalb and colleagues, 2008–2010).

Progress was measured on shared benchmarks: the PASCAL Visual Object Classes challenge (2005–2012) and then ImageNet (2009), millions of images whose labels were checked by crowd workers on Amazon Mechanical Turk. In the ImageNet challenge, which began in 2010, the winning top-5 error was 28.2% in 2010 and 25.8% in 2011, both with features designed by hand, and 15.3% for AlexNet in 2012 (Figure 2j). Learned models without deep networks also shipped: Microsoft's Kinect (2010) labeled every pixel of a depth image with a body part using randomized decision forests trained on synthetic images.

And much of vision was geometry, such as reconstructing buildings in three dimensions by matching features across thousands of tourists' photographs (Photo Tourism, 2006).

B.5 Language and speech

For language processing, the 1990s were a statistical revolution. IBM's statistical approach to machine translation (1990) learned word alignments and translation probabilities from the English and French proceedings of the Canadian parliament (Brown and colleagues, 1993). Annotated corpora such as the Penn Treebank (1993) made it possible to learn parsers (Collins, 1997; Charniak, 2000) and taggers (maximum entropy models, 1996; conditional random fields) from examples instead of writing grammars by hand. Phrase-based translation (Koehn, Och, and Marcu, 2003) translated short sequences of words at a time, with an n-gram language model choosing fluent output. Google's own statistical translation went live in 2006 and was replaced by a neural system in 2016.

The n-gram models of Week 2 were the language models of this period, with Kneser–Ney smoothing (1995) and its refinements (Chen and Goodman, 1999) as the state of the art. Scale was already a main lever. A 2007 paper from Google titled *Large Language Models in Machine Translation* trained n-gram models on up to two trillion tokens, with up to 300 billion distinct n-grams; translation kept improving with more data, and with enough data a crude smoothing method did about as well as Kneser–Ney. *The Unreasonable Effectiveness of Data* (Halevy, Norvig, and Pereira, 2009) drew the general lesson. Word meaning was approached through distribution (§6): latent semantic analysis (1990) reduced a matrix of word counts in documents by the singular value decomposition, Brown clustering (1992) grouped words by the words around them, and neural models learned word vectors together with a predictor (Bengio and colleagues, 2003; Collobert and Weston, 2008; Collobert and colleagues, 2011; the recurrent language model of Mikolov and colleagues, 2010). IBM's question-answering system Watson, which combined many statistical components, won a televised *Jeopardy!* match against two champions in 2011.

Speech recognizers modeled the sounds of each word with hidden Markov models whose states emit acoustic features through mixtures of Gaussians. Continuous dictation, without pauses between words, reached personal computers in 1997 with Dragon NaturallySpeaking.

B.6 Robots and self-driving cars

Ernst Dickmanns's group at the Bundeswehr University Munich drove cars on public roads by computer vision. In 1994 their Mercedes, VaMP, drove more than 1,000 km in normal traffic on highways near Paris, changing lanes on its own, and in 1995 it drove about 1,600 km to Denmark with about 95% of the distance autonomous (Dickmanns, 1997). At Carnegie Mellon, Dean Pomerleau's ALVINN (1989) was a small neural network that mapped a low-resolution camera image to a steering direction. A later version learned by watching a human driver and could learn to steer the Navlab van in minutes, an early case of learning by imitation. In No Hands Across America (1995), Navlab 5 steered itself for 2,797 of the 2,849 miles from Pittsburgh to San Diego, with a vision system that was not a neural network and with people working the throttle and brakes.

The DARPA Grand Challenges turned autonomy into a race. In 2004 no vehicle finished the desert course of about 142 miles, and the best covered about 7. In 2005, Stanford's Stanley finished a 132-mile course in under seven hours, and four other vehicles also completed it, one of them over the time limit. In the 2007 Urban Challenge, on a mock city course with other traffic, Carnegie Mellon's Boss won. These systems were built on probabilistic robotics (Thrun, Burgard, and Fox, 2005): laser range finders and cameras, Kalman and particle filters for estimating the car's position, maps, and planners, with learning in parts of perception. Stanley, for example, used its laser to find drivable ground close by and trained a classifier on the camera image to see the road farther ahead. Google started its self-driving project in 2009 with veterans of these races. Robots also reached homes and other planets: the Roomba vacuum cleaner (2002), and NASA's Mars rovers Spirit and Opportunity (2004), which picked their own paths around hazards.

B.7 The revival

Neural networks never disappeared. The convolutional networks of LeCun, Bottou, Bengio, and Haffner (1998) read handwritten digits; a check reader built on them was deployed from 1996 and, by LeCun's account, read over a tenth of the checks written in the United States by the end of the decade. Long short-term memory (Hochreiter and Schmidhuber, 1997) gave recurrent networks a path along which gradients do not vanish, and Bengio's neural language model (2003) learned word vectors (§6). From

2004, a program of the Canadian Institute for Advanced Research led by Geoffrey Hinton kept a small community together. In 2006, Hinton and colleagues showed that deep networks could be trained by first training them one layer at a time without labels (Hinton, Osindero, and Teh, 2006; Hinton and Salakhutdinov, 2006), and the phrase "deep learning" came into wide use.

Then three things changed together. **Data**: the web, digital photography, and crowdsourcing produced labeled collections such as ImageNet, whose challenge set had 1.2 million training images. **Computation**: graphics processors became programmable for general computation with NVIDIA's CUDA (2007), and Raina, Madhavan, and Ng (2009) trained deep networks up to 70 times faster on them than on a dual-core processor. **Technique**: rectified linear units (Nair and Hinton, 2010; Glorot, Bordes, and Bengio, 2011), careful initialization (Glorot and Bengio, 2010), and dropout (2012) made it possible to train deep networks directly from labeled data, without the layer-by-layer start.

The first decisive results came in speech. Between 2009 and 2012, deep networks replaced the Gaussian mixtures of speech recognizers. On a benchmark of conversational telephone speech the word error rate fell from 27.4% to 18.5% (Seide, Li, and Yu, 2011); four research groups reported similar gains (Hinton and colleagues, 2012); and Google's voice search on Android switched to a deep network in 2012. In vision, convolutional networks trained on graphics processors recognized traffic signs more accurately than people in 2011 (0.54% error against 1.16%), and a network trained without labels on frames from ten million YouTube videos developed a unit that responds to cats (2012). Later in 2012, AlexNet, trained for five to six days on two consumer graphics cards, won the ImageNet challenge with 15.3% top-5 error against 26.2% for the next entry (§1, item 9). Within two years the leading entries were all deep convolutional networks, and the methods spread to translation, games, and, by way of the Transformer, to the language models of this course.

The two transitions of this appendix are the bitter lesson of §4 at work twice. In the 1990s, rules written by hand gave way to statistics estimated from data; after 2012, features designed by hand gave way to features learned from data.